

Tecnología Meta4

Manual de desarrollo development framework



© 2013 Meta4 Spain, S.A. Se reservan todos los derechos.

AVISO: Este manual está protegido por la legislación referente a propiedad intelectual e industrial y por tratados internacionales. La utilización permitida de esta documentación queda limitada a su uso en conexión con el producto, y todo uso no autorizado será perseguido de acuerdo con la legislación aplicable. Se prohíbe su copia, modificación, reproducción o distribución sin permiso del titular.

Meta4 PeopleNet © 1999 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 KnowNet © 1996 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 e-mind © 2001 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 PeopleNet Ksystem © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 t.innova © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4[®], Meta4Mind[®], Meta4 PeopleNet[®], Meta4 KnowNet[®], Meta4 e-mind[®], Meta4 PeopleNet Ksystem[®] y Meta4 t.innova[®] son marcas registradas propiedad de Meta4Spain, S.A.

Otros nombres de compañías, productos o servicios son marcas registradas o nombres comerciales de sus respectivos propietarios.

Meta4 Spain, S.A.
Centro Europa Empresarial
Edificio Roma
C/ Rozabella, 8
Ctra. de La Coruña, km 24,200
28290 Las Rozas, Madrid
ESPAÑA
<http://www.meta4.com>

Fecha de creación: julio de 2006.

Fecha de la última publicación: diciembre de 2013.

Tabla de contenidos

1 Introducción

■ Acerca de este manual	1
Propósito	1
Audiencia	1
Requisitos	2

2 ¿Qué es el development framework?

■ Acerca de este capítulo	3
■ El Development Framework	3
■ Cómo acceder al framework	4
■ Plantillas	6
Plantilla de filtro	7
Plantilla de lista y mantenimiento	12
Plantilla de multinodo	16
Plantilla de formulario de información	17
Plantilla de petición de parámetros y ejecución de procesos	19
Plantilla de tablas de valores	20
Plantilla de calendario	32
Plantilla de desktop	34
■ Exportación/Importación desde Excel	39
Ejecución de procesos de importación y exportación	40
■ Metodología	42
Nomenclatura	42
Reglas generales	44
Traducciones	46

3 Caso Práctico

■ Acerca de este capítulo	49
■ Creación de una lista y mantenimiento	50
■ Creación de un multinodo	53
■ Creación de un formulario de información	56
■ Creación de un filtro	60
■ Creación de una tabla de valores	63

■ Creación de un asistente de petición de parámetros y ejecución de procesos.	67
■ Creación de un calendario en una presentación.	81
■ Creación de un desktop.	84

4 Configuración avanzada

■ Acerca de este capítulo	87
■ Estructura y funcionamiento de las plantillas	87
■ Metodología de desarrollo avanzada (sin herramientas)	90
■ Transferencia de datos entre Meta4Objects	93

5 Caso práctico avanzado

■ Acerca de este capítulo	101
■ Añadir tabla histórica al producto	101
■ Mantenimiento del periodo	102
■ El alta	105
■ La baja (parametrización)	107
■ La cancelación de la baja (parametrización)	108
■ Borrado (parametrización)	108

Introducción

Acerca de este manual

Propósito

El objetivo es presentar un enfoque práctico sobre el uso del development framework para la creación o modificación de desarrollos funcionales en PeopleNet utilizando tecnología PeopleNet. La estructura del contenido es la siguiente:

- Capítulo 1: ¿Qué es el development framework?. Este capítulo se considera de obligada lectura si no se ha trabajado previamente con el mismo. En él se detallan temas como:
 - Definición formal del framework y las ventajas que conlleva su uso. Puntos de acceso desde PeopleNet.
 - Tipología de plantillas
 - Metodología de carácter general
- Capítulo 2: Caso práctico de creación y modificación de los objetos más habituales.
- Capítulo 3: Consideraciones de carácter avanzado sobre configuración y funcionamiento de ciertos objetos y procesos del framework.
- Capítulo 4: Casos prácticos funcionales avanzados donde se utilizarán todos los conceptos explicados previamente.

Audiencia

Este manual está dirigido a desarrolladores e implantadores de funcionalidad en PeopleNet.

Requisitos

Para un correcto uso y entendimiento del contenido del manual es necesario tener conocimientos de algunos conceptos tecnológicos: arquitectura Meta4Objects, herencia, etc.

Para el correcto entendimiento de la configuración de tablas de valores conviene tener formación básica en nómina.

¿Qué es el development framework?

1

Acerca de este capítulo

En este capítulo se detalla qué es el development framework, qué ventajas tiene su uso y cómo se puede acceder al mismo desde PeopleNet. Posteriormente se explicarán las plantillas o elementos funcionales principales que lo conforman y finalmente se verá la metodología básica de desarrollo que propone.

El Development Framework

El development framework es una metodología para facilitar al desarrollador la creación o modificación de funcionalidades del producto y la configuración del mismo. Su uso presenta las siguientes ventajas:

- Modelo de desarrollo común: el framework ofrece una serie de reglas y prácticas de desarrollo que garantizan que las nuevas funcionalidades sigan un modelo común y los estándares Meta4, lo cual mejora la usabilidad y mantenimiento del producto.
- Rapidez de desarrollo: las herramientas y metodologías simplifican el esfuerzo y el tiempo de implementación de los desarrolladores.
- Mantenimiento: el uso de últimas tecnologías en los componentes que se generan facilitan de forma sustancial el mantenimiento de los desarrollos.

El framework incorpora unas plantillas que permiten resolver de forma común el desarrollo de las funcionalidades más habituales que se implementan en el producto: mantenimiento de datos, filtros, listas, formularios de información, etcétera. Las plantillas están compuestas principalmente por Meta4Objects y presentaciones que sirven como base de la metodología para la creación y modificación de funcionalidad.

Además, el framework contiene herramientas que permiten generar o modificar de forma automática gran parte de la funcionalidad. Las características principales de estas herramientas son:

- Generan objetos de estructura coherente con las nuevas plantillas, utilizando tecnología de herencia de componentes e includes.
- Ciclo de vida: tras generar un objeto, éste puede ser editado en cualquier momento utilizando las mismas herramientas.
- Las herramientas se encuentran totalmente integradas con las herramientas de diseño tradicionales de los objetos que generan; si se modifican los objetos con los diseñadores clásicos, posteriormente pueden editarse a través de las herramientas framework.

Cómo acceder al framework

Existen diferentes formas de tener acceso a las herramientas del framework:

- Desde opción de menú **Herramientas de desarrollo | Desarrollar con el development framework**
- Desde las herramientas de desarrollo tradicional:
 - *Diseñador de Meta4Objects:*
 - al abrir un Meta4Object bien de tipo Multinodo o bien de tipo lista y mantenimiento
 - al crear un nuevo Meta4Object y seleccionar uno de los tipos indicados
 - *Diseñador de presentaciones:*
 - al crear una nueva presentación y seleccionar uno de los siguientes tipos: Mantenimiento, Multinodo o Formulario de información.
 - al abrir una presentación de uno de estos tipos y utilizar la opción de menú Herramientas | Asistente del development framework.
 - *Diseñador del modelo de datos:*
 - desde la opción de menú **Herramientas | Asistente del development framework**



Como puede acceder a las herramientas del framework desde varias opciones para crear o modificar Meta4Object, tenga en cuenta que en el caso de que lo que desee es hacer una modificación de un Meta4Object aparecerá una ventana de aviso que le advierte de la posibilidad de modificar dicho Meta4Object desde la herramienta del framework o desde el Diseñador de Meta4Objects. Es importante que recuerde que el impacto si usa el framework será que los cambios que efectúe sobre la estructura del Meta4Object se propagarán a la presentación asociada, es decir si elimina o añade algún elemento este se eliminará o se añadirá cuando la herramienta acceda al

diseñador de presentaciones. De manera inversa si usa el diseñador de Meta4Objects para la modificación los cambios que efectué sobre el Meta4Object no se propagarán a la presentación asociada, así cuando abra el Meta4Object desde el diseñador de presentaciones no se verán reflejados los cambios realizados.

Plantillas

El framework incorpora una colección de plantillas; cada una de ellas representa un patrón básico de funcionalidad que se repite varias veces en el producto. Las plantillas están basadas en las nuevas tecnologías de includes y herencia así como en la incorporación de nuevas propiedades y nuevos comportamientos de los objetos visuales actuales. Las plantillas están formadas principalmente por Meta4Objects y presentaciones y la metodología propone usar la herencia para crear nuevos objetos o modificar los existentes.

Las funcionalidades de referencia de la colección de plantillas del framework son las siguientes:

- **Filtros y listas:** objeto que permite al usuario introducir criterios para buscar un número de registros y facilitar la selección de uno o varios de ellos. Suele ser un paso previo para tener acceso al resto de funcionalidad.
- **Mantenimiento de datos:** objeto para insertar o actualizar datos mientras se examinan otros registros de la misma tabla. Permite la ejecución de acciones u enlaces a otros objetos.
- **Formularios de información:** objeto para visualizar información descriptiva de un proceso.
- **Petición de parámetros y ejecución de procesos:** objeto para la recogida ordenada de parámetros en varios pasos que luego se aplicarán a la ejecución de un proceso de usuario como un informe, una planificación de una tarea, una salida a carta de Word, etcétera.
- **Tablas de valores:** objeto para editar valores de elementos de nómina en formato matriz tridimensional. Existen tres tipos de tablas de valores: valores fijos, valores incidencias y entrada masiva de incidencias.
- **Calendario:** objeto para mostrar información en formato calendario. Un ejemplo es el calendario de incidencias de una persona en el último año.
- **Desktops:** objeto que permite tener accesible desde una misma pantalla los recursos de trabajo más habituales: menús, gráficos, informes, poblaciones, procesos de negocio y lista de tareas.

el desarrollo de cada una de las funcionalidades siguiendo la plantilla correspondiente dará lugar a dos tipos de objetos definidos propios de arquitectura del producto de Meta4:

- *Presentación:* objeto que define la visualización de los datos en pantalla, elementos a mostrar y formato de los mismos.
- *Meta4Object:* objeto que define la estructura de los datos con los que se va a trabajar, así como las operaciones que se pueden realizar con los mismos.

Dependiendo de la plantilla seleccionada, habrá casos en los que exista una herramienta que permita la generación y modificación automática de cada objeto, en cuyo caso se describirá el uso de la misma, y en caso de que no exista se describirá el proceso manual de generación de los mismos.

Las nuevas plantillas presentan una serie de capacidades avanzadas para facilitar su uso y el mantenimiento:

- Posicionamiento automático de los campos de edición de un formulario al añadir o borrar otros.
- Panel de selección de registros ajustable de forma automática por filas y columnas. La cabecera de la tabla agrupa en una única columna los campos ID y nombre correspondiente. Según la plantilla podrá ser de solo lectura (no modificable) y se resalta en color toda la fila seleccionada.
- En listas y carriles se mejora el aspecto presentando filas en dos colores y ajustando de forma automática los campos a lo ancho.
- Nuevos paneles de enlaces y acciones para facilitar la navegación entre presentaciones.
- Visualización y ocultación de paneles de forma automática dependiendo de la existencia o no de registros.

A continuación se describen cada una de las plantillas mencionadas indicando:

- objetivo principal y funcionalidad que ofrece
- pantalla asociada y estructura de la misma
- ejemplo



Las plantillas que son usadas por las diferentes herramientas son objetos tecnológicos y no son modificables.

Plantilla de filtro

La plantilla de filtro facilita la introducción, a través de un formulario QBF (Query by Form), de un criterio de búsqueda. El resultado de esa búsqueda se mostrará normalmente en una lista y servirá para dar acceso al usuario a una funcionalidad determinada como es un mantenimiento de datos, una tabla de valores, etcétera. Estos tipos de filtros son, normalmente, invocados de forma automática por el sistema al ejecutar la funcionalidad asociada o invocados por el usuario pulsando el botón Lista de la barra de herramientas. Estos filtros se conocen como filtros tipo lista y son filtros que sólo se aplican sobre los nodos raíz de los Meta4Objects de forma que optimizan la carga de objetos complejos con múltiples nodos.

Existen otros tipos de filtro llamados filtros de nodo, aplicables sobre cualquier nodo de un Meta4Object y que permiten dos cosas:

- Filtrar inicialmente el número de registros a tratar en un mantenimiento, formulario de información, etcétera. La diferencia con el filtro tipo lista radica en que el sistema carga la información de todos los registros que cumplen el criterio y una vez establecido no se puede volver a invocar a no ser que se vuelva a cargar el formulario. Además este tipo de filtro permite hacer cambios en varios registros y navegar a otros sin guardar los mismos. Los filtros se pueden introducir de dos maneras:
 - Según un criterio de búsqueda establecido en una filtro QBF
 - Según un criterio de búsqueda establecido en una filtro QBF y, a continuación, una selección de registros en una lista múltiple.
- Filtrar datos ya cargados en una pantalla. En este caso, el desarrollador funcional establece el momento y lugar donde se invoca al filtro, por ejemplo, al pulsar un botón en una pantalla determinada. En ese caso, y según su codificación gestionará la llamada al filtro y recibirá al finalizar el resultado del mismo en Api SQL que tratará principalmente para realizar una nueva carga de datos en pantalla o hacer otra operación a medida.

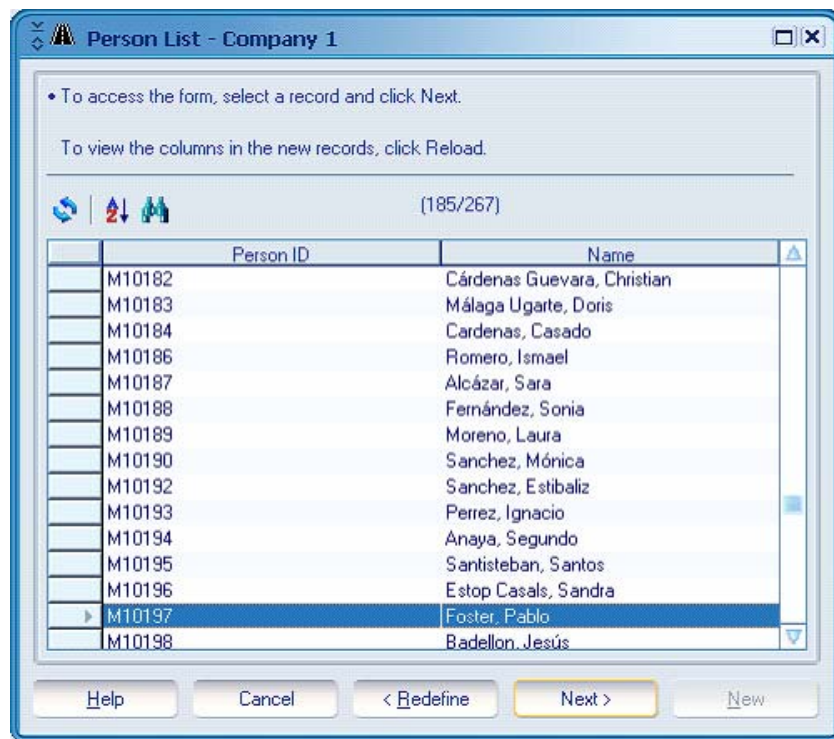


Se puede usar letras mayúsculas o minúsculas indiscriminadamente en los criterios de búsqueda que decida al usar un filtro. La búsqueda no será sensible a estos caracteres.

Visualmente los filtros pueden presentar las siguientes apariencias:

Filtros tipo lista

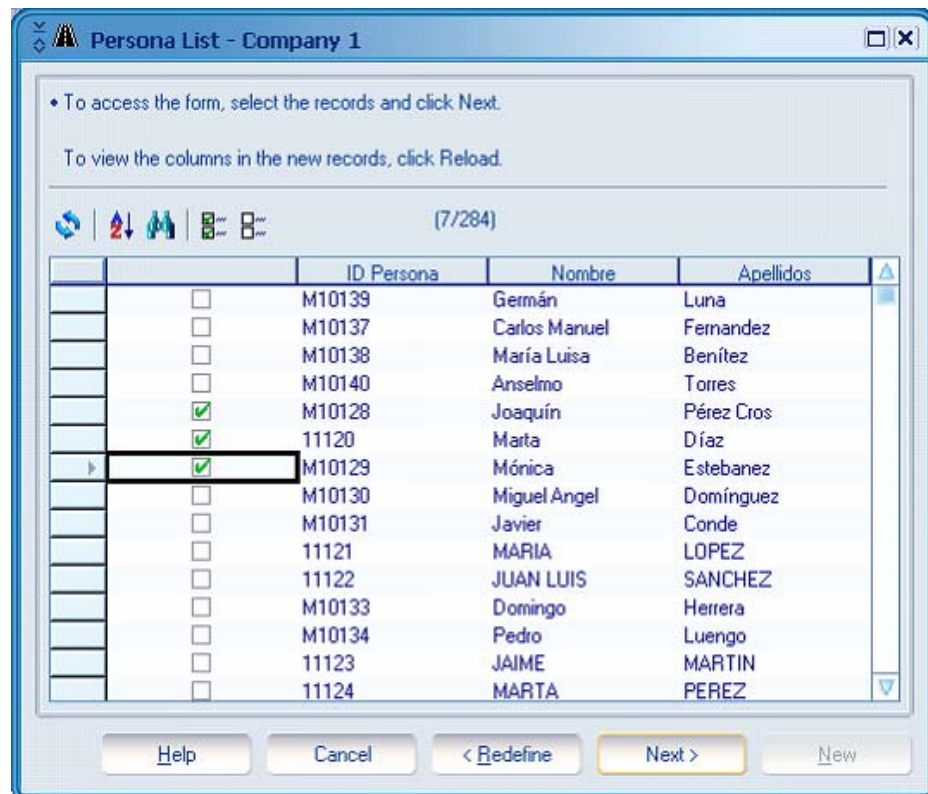
Esta permite al usuario introducir un criterio de búsqueda. Normalmente preceden a ventanas de filtro tipo lista (sencilla y múltiple) pero también pueden ser llamados desde cualquier acción de usuario si el desarrollador así la ha programado. Si lo que se desea filtrar no tiene demasiados registros no es obligatorio y se pueden mostrar directamente las siguientes ventanas.



Ventana filtro tipo lista

Suele aparecer una vez que el usuario ha rellenado un criterio de búsqueda en la ventana QBF mostrando el resultado de esa búsqueda. Como se ha dicho anteriormente se accede a la misma por el botón Lista de la barra de herramientas o al entrar de forma automática en ciertos componentes (si el desarrollador así lo configuró). Permite:

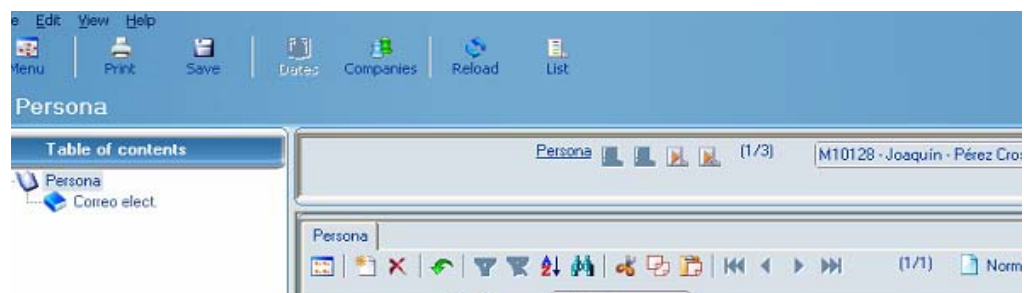
- Seleccionar fácilmente un registro.
- Buscar, ordenar o recargar los registros en pantalla para facilitar la selección de uno de ellos.
- Una vez seleccionado uno, el sistema permitirá la navegación por el resto.



Ventana filtro tipo lista multiselección

También aparece normalmente una vez que el usuario ha rellenado un criterio de búsqueda en la ventana QBF. Permite:

- Seleccionar de forma múltiple un conjunto de registros a tratar.
- Buscar, ordenar o recargar los registros en pantalla para facilitar la selección de un conjunto de ellos.
- Seleccionar o deseleccionar de forma múltiple
- Una vez seleccionado un conjunto, el sistema sólo te permitirá navegar por ese conjunto.



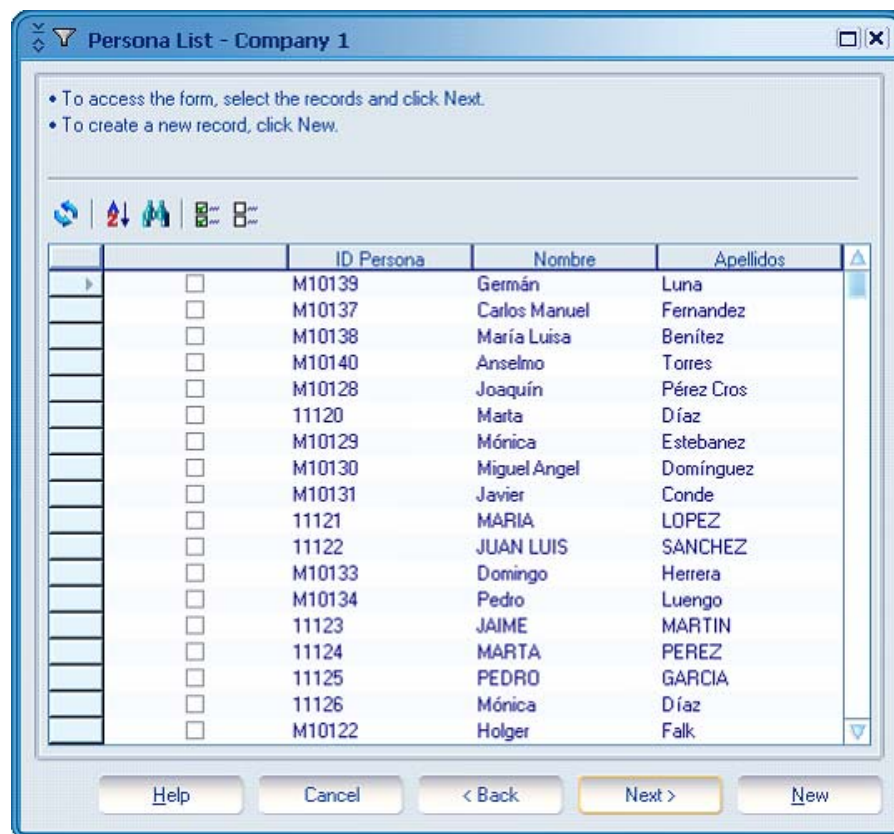
Ventana de navegación por registros de filtro tipo lista y botón de acceso al mismo

Filtros de nodo

Su objetivo es el mismo que los filtros tipo lista pero sólo pueden aplicarse una vez al cargar el formulario. La ventaja principal frente a los otros tipos es que permite hacer cambios en varios registros y poder navegar a otros sin guardar los mismos. Sin embargo sólo pueden invocarse inicialmente y traen toda la información seleccionada. Se pueden aplicar sobre cualquier nodo de un Meta4Object.

Este tipo de filtros se presenta de dos formas distintas:

- Mostrando una única ventana filtro QBF por cada nodo. El aspecto de la ventana filtro QBF es exactamente igual que el que la ventana QBF del filtro tipo lista.
- Mostrando una ventana filtro QBF y, a continuación, una selección de registros en una lista múltiple. En este caso la diferencia visual con el la lista múltiple selección del filtro tipo lista es el icono de la cabecera de la ventana.



Plantilla de lista y mantenimiento

Esta plantilla contiene la funcionalidad básica para crear componentes de tipo lista y mantenimiento.

Para crear y modificar componentes basados en esta plantilla existe un asistente en el framework que automatiza estos procesos. Para acceder a ella de forma directa utilice las opciones de menú:

"Herramientas de desarrollo | Desarrollar con el development framework | Crear lista y mantenimiento

"Herramientas de desarrollo | Desarrollar con el development framework | Modificar lista y mantenimiento.

Lista

Una lista es una ventana que facilita la selección de datos con el objetivo de

cumplimentar campos. Se despliegan normalmente desde los botones de lista asociados a los campos:

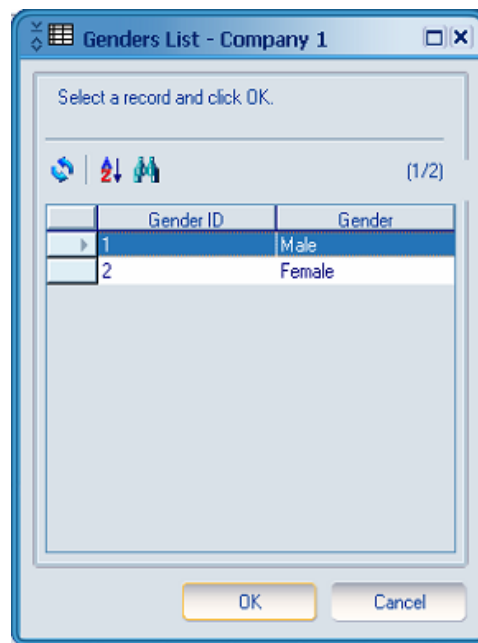


Country ID: 034 Spain

Gender ID: 1 Male

Title ID: 01 Mr.

Su aspecto visual es el siguiente:



La lista también puede tener ventana de filtro QBF para filtrar el número de registros si el desarrollador considera que devolverá muchos registros.

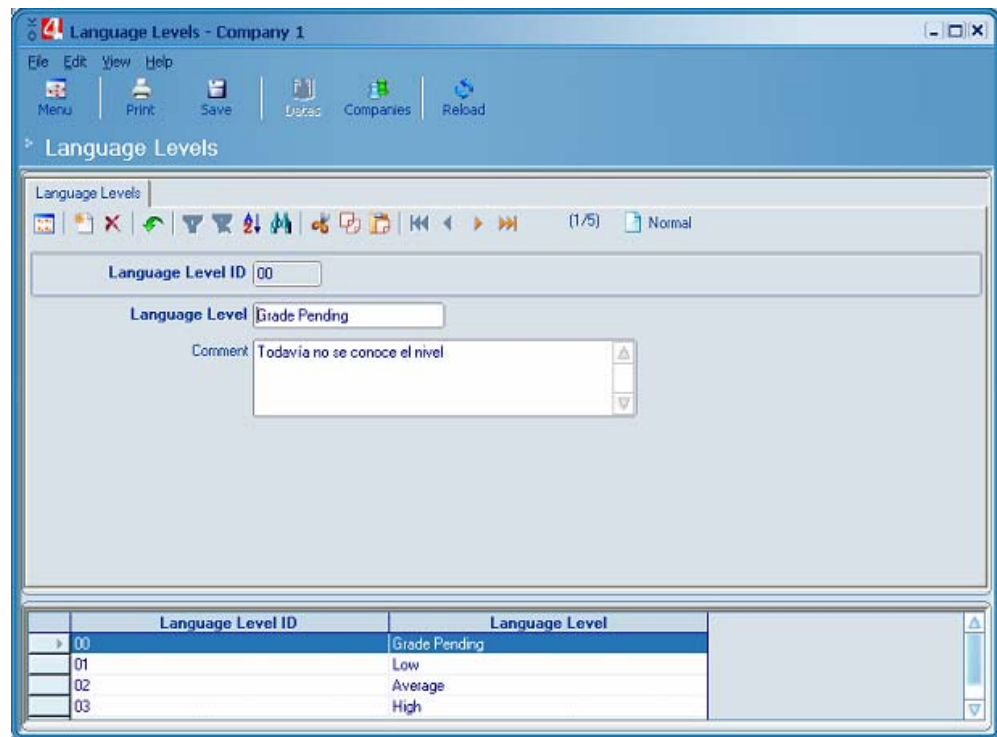


Puede añadir iconos en columnas de una ventana tipo lista. Esto puede facilitar la selección de datos desde las listas que lo tengan configurado.

Mantenimiento

Esta plantilla tiene como objetivo el mantenimiento de datos ya sea a nivel inserción, actualización o borrado. Un claro ejemplo son las tablas de referencia.

El aspecto visual habitual suele ser el siguiente:



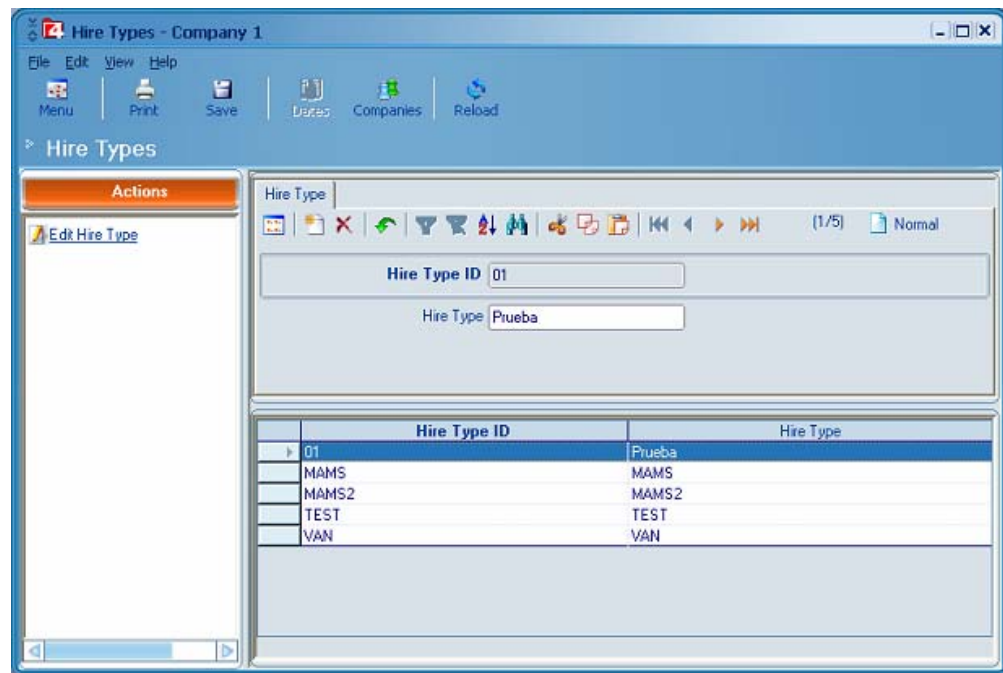
En la parte inferior existe una tabla para navegar por todos los registros existentes y en la parte superior un formulario de edición de los datos. Con los botones de la barra de herramientas se permite

- Añadir nuevos registros
- Borrar existentes
- Ordenar alfabéticamente la tabla y buscar registros, además de las facilidades habituales de copiar, pegar, etcétera.
- Navegar por los distintos registros.

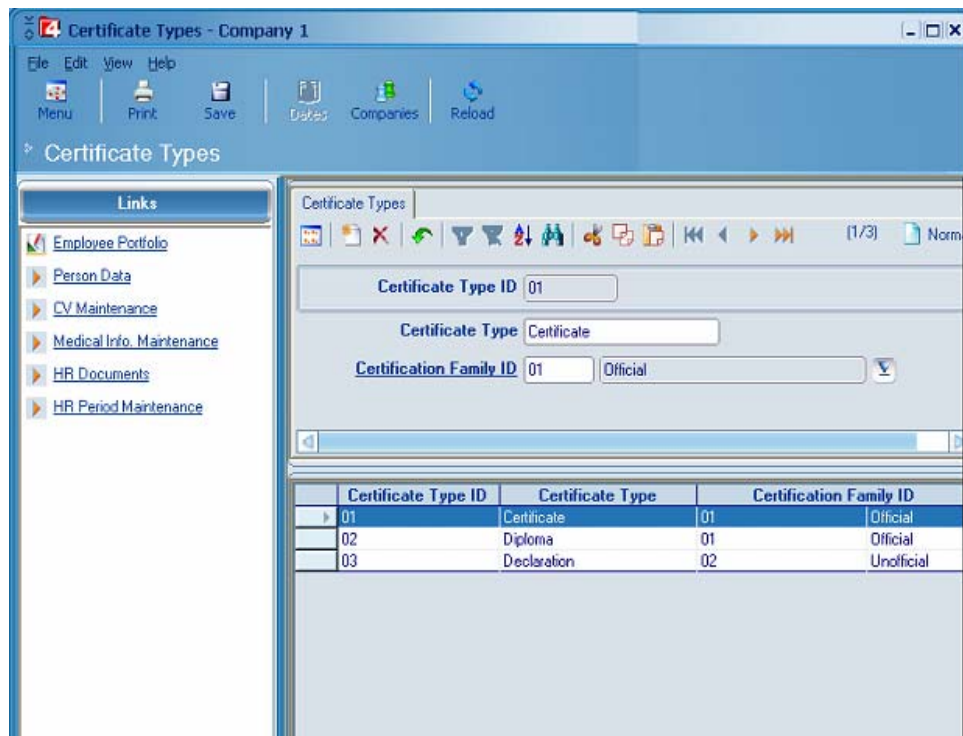
A esta pantalla se puede tener acceso a través de un filtro tipo lista o un filtro de nodo. Dependerá de si el desarrollador considera que la tabla de referencia puede tener muchos registros.

El framework incorporará un panel a la izquierda si el desarrollador ha metido enlaces a otros componentes o permite acciones del tipo ejecución de distintos métodos. El aspecto sería:

Con acciones:



Con enlaces:



Plantilla de multinodo

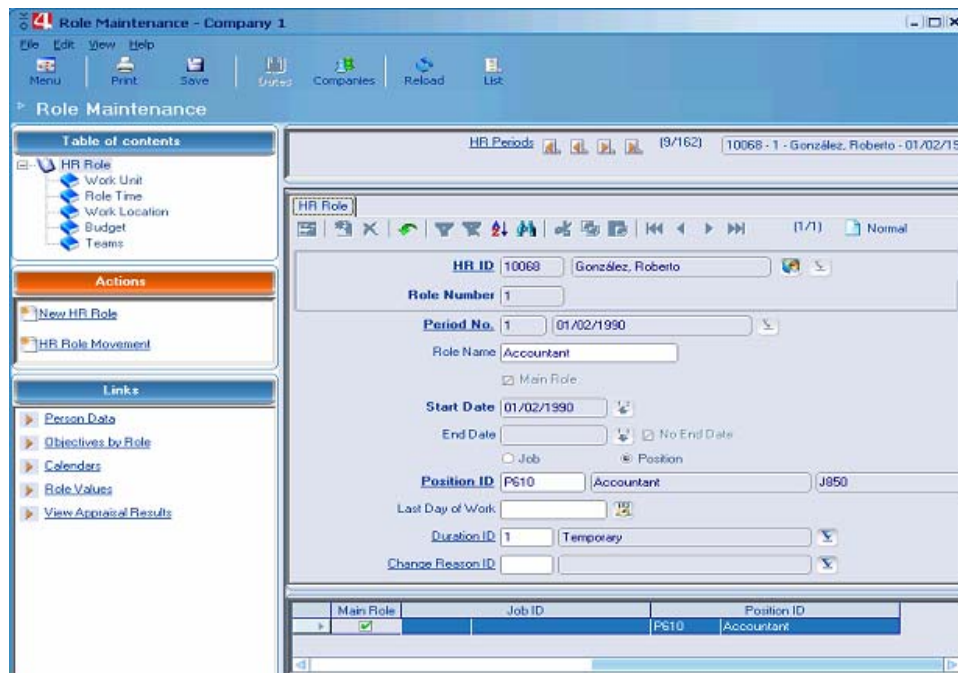
La plantilla de multinodo tiene como objetivo el mantenimiento de datos de estructuras de datos complejas. Por ejemplo, el mantenimiento de la persona, del periodo o del rol. En estos casos el multinodo suele venir filtrado por un filtro tipo lista que restringe la población de trabajo y evita tener cargados en memoria múltiples bloques de información pertenecientes a todos los registros sobre los que se quiere trabajar.

Al igual que con los mantenimientos, para crear y modificar componentes basados en esta plantilla existe un asistente en el framework que automatiza estos procesos. Para tener acceso a ella de forma directa utilice las opciones:

Herramientas de desarrollo | Desarrollar con el development framework | Crear multinodo

Herramientas de desarrollo | Desarrollar con el development framework | Modificar multinodo.

El aspecto visual más habitual de un multinodo es el siguiente:



Estas presentaciones permiten:

- Insertar, actualizar y borrar información de los distintos nodos permitiendo navegar por la estructura del multinodo utilizando el árbol situado en la parte superior izquierda.
- Por cada nodo, permite las mismas operaciones descritas en un mantenimiento. En la parte derecha se tendrá un formulario de edición de datos y una tabla para navegar por los distintos registros. Esta tabla no aparece en el nodo raíz si solo hay un registro en el mismo. La barra de herramientas permite las mismas operaciones descritas anteriormente.
- Navegar a otros componentes como formularios de información u otros mantenimientos mediante el panel de enlaces.
- Ejecutar acciones: el framework incorpora un panel bajo el de enlaces si el desarrollador ha permitido ejecutar acciones del tipo ejecución de distintos métodos.

Plantilla de formulario de información

Esta plantilla tiene como objetivo la creación de presentaciones que sirvan como consulta de datos. Por ejemplo, para consultar información básica sobre una persona tenemos el formulario de información Información de la persona. Otros formularios de información son la Información médica o Datos del CV.

El aspecto de un formulario de información es el siguiente:

Person Information - Company 1

File Edit View Help

Menu Print Save Tables Companies Reload List

> Person Information

Other information

- Applicant Information
- Period Information
- Role Information
- C.V. Information
- Medical Information

Person 10063 - Gutierrez García, Alejandro

Applicant - Employee

Date of Birth 05/02/1967 Web Page

Phone Number Mobile Phone

E-mail Address

Person

County of Birth Spain County of Birth Town of Birth

Gender Male Title ID

Bank Data

Bank Branch ID	Entitled	Account Number	Start Date
BANK_05 Banco Santander Sucursal 28 HLD002	Alejandro Gutierrez	34093434	01/02/1990

Home Addresses

Address	Location
C/ San Angela de la cruz Nº 27 2º España Madrid 28000	Residence

Phone/Fax

Line Type ID	Location	International Area Code	Country Code	National Area Code	Phone Number
	34	1	91	58 36565	

Marital Status

Marital Status	Start Date	End Date
Married	01/02/1990	

Links

- Person Maintenance

Estas presentaciones permiten:

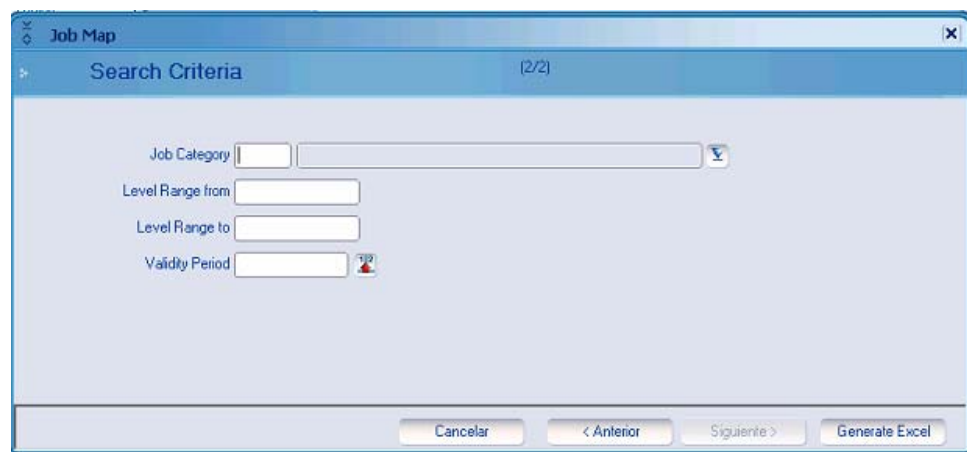
- Consultar información: muestran normalmente una cabecera con la información principal y una serie de formularios y tablas con la información que se desea mostrar. Si la información a mostrar es 1:1 respecto a la principal se debe mostrar en formularios y si es 1:N en tablas. La información principal se mantiene fija en el formulario y el resto se puede mover con la barra de desplazamiento vertical.
- Navegar a mantenimientos o multinodos mediante enlaces y a otros formularios de información.
- Normalmente el acceso a los formularios de información se realiza a través de un filtro tipo lista. Esto permite tener acceso a la lista en cualquier momento y navegar por cualquier elemento de la población filtrada teniendo en memoria la información básica.
- Para crear componentes basados en esta plantilla existe un asistente en el framework que automatiza este proceso aunque ciertas acciones hay que realizarlas de forma manual. En un capítulo posterior se verá con más detalle este proceso.

Para acceder al asistente de forma directa utilice la opción de menú:

Plantilla de petición de parámetros y ejecución de procesos

La plantilla de petición de parámetros y ejecución de procesos permite recoger información que el usuario introduce en una serie de pasos y finalmente ejecuta un proceso utilizando toda la información suministrada.

Los tipos de procesos permitidos son:



Ejecución de un informe.

- Ejecución de una tarea a través del planificador de tareas.
- Ejecución de un proceso de combinación de correspondencia (mail merge) de Microsoft Word.
- Ejecución de un método de un Meta4Object.

Las características principales de la plantilla referentes a la petición de parámetros son:

- Pasos obligatorios y no obligatorios. Los pasos de un proceso se pueden definir como obligatorios, de modo que sea necesario introducir los datos para poder ejecutar el proceso final (por ejemplo, un informe).
- Número de pasos variable. La plantilla provee de mecanismos para que el desarrollador permita mostrar o no ciertos pasos en ejecución según el usuario vaya seleccionando datos.

- **Filtro dinámico.** La plantilla permite mostrar el filtro dinámico previo a la ejecución del proceso. De esta forma puede utilizar la ventana actual del editor de sentencias para componer un filtro y aplicarlo en la ejecución del proceso. Aquí también se parametrizan los Action Set, incluyendo los valores en el include, de la misma forma que para parámetros globales de la presentación.

Como ejemplo de uso de esta plantilla tenemos **Organización | Análisis | Mapa de puestos** que muestra una nivelación de los puestos de su organización clasificados a elección del usuario por Unidad organizativa o por Clase interna de puesto, en una salida gráfica de Microsoft Excel.

Como se ha dicho anteriormente, la plantilla suministra el marco para crear una serie de pasos donde el usuario debe introducir la información y finalmente aplicarla en la ejecución de un proceso. Por tanto, un componente basado en esta plantilla tendrá varios formularios, tantos como pasos se deseen, aunque todos tendrán la misma estructura con tres partes bien diferenciadas:

- La parte superior tiene el nombre del paso y su orden respecto al número total de pasos. También permite incluir un texto descriptivo largo para facilitar al usuario las instrucciones relativas a los parámetros a cumplimentar en ese paso (en el ejemplo de arriba no se ve).
- La parte intermedia contiene los campos o parámetros propios del proceso a ejecutar y que el usuario debe rellenar.
- La parte inferior tiene cuatro botones: Cancelar para cancelar el proceso, Anterior y Siguiente para navegar entre los pasos y Ejecutar para ejecutar finalmente el proceso. La etiqueta de este botón puede modificarse para adaptarse al proceso particular.
- Para crear componentes basados en esta plantilla se deberá proceder de forma manual pues el framework no suministra asistente que facilite su creación. En un capítulo posterior se realizará un ejemplo completo y en el apéndice se verá el detalle de todas las posibilidades de configuración.

Plantilla de tablas de valores

La plantilla de tablas de valores permite asignar valores a los elementos de nómina desde el punto de vista de la organización o del individuo en un periodo de tiempo determinado.

Las tablas de valor pueden clasificarse por su contenido en tres grandes grupos:

Valores fijos. Son tablas que incluyen los valores de elementos de nómina asignados a nivel organizativo o a individual para un periodo o rol del RH pero que son válidos durante más de una paga. Es el caso de los valores asignados

por empresa, unidad organizativa, lugar de trabajo, valores fijos por periodo, etcétera.

Inserción masiva de valores fijos. Son tablas que permiten la asignación masiva de valores a elementos de nómina de carácter individual válidos durante varias pagas.

Valores de incidencias. Son tablas que incluyen los valores de elementos de nómina asignados a nivel individual válidos para una sola paga. Es el caso de los valores incidencias asignados por periodo de RH o rol de RH.

Inserción masiva de valores de incidencias. Son tablas que permiten la asignación masiva de valores a elementos de nómina de carácter individual válidos para una sola paga.

Por tanto existen cuatro plantillas para el desarrollo de estas funcionalidades con sus correspondientes presentaciones.

Adicionalmente, estas plantillas utilizan una serie de formularios auxiliares como:

- Selección de elementos de nómina
- Valores por defecto
- Edición por clave primaria, pivotado o de proyección
- Histórico a fecha de pago y a fecha de imputación
- Tramos

Para crear tablas de valores no existe herramienta automática que facilite el proceso. Será necesario seguir la metodología paso a paso de forma manual y utilizar las herramientas de desarrollo clásicas (Diseñador de Meta4Objects y Diseñador de Presentaciones). En un capítulo posterior veremos como crear un ejemplo de cada uno de los tipos y en el apéndice se detallan todas las opciones de configuración.

Valores fijos

Su función principal es la de facilitar la asignación de valores para periodos superiores al de una paga. Como ejemplos más representativos tenemos los valores de empresa, de unidad organizativa y de lugar de trabajo.

El aspecto general de este formulario es el siguiente:

Este formulario se puede dividir en tres zonas bien diferenciadas:

- La parte superior izquierda es la zona de filtro donde el usuario establecerá las fechas de imputación y de pago por las que se filtrarán los valores al cargar la tabla y sobre los que se desea asignar valores. La primera vez que se entra en el formulario, dichas fechas se corresponden con el primer día de la primera paga abierta y no bloqueada.
- La parte superior derecha presenta una serie de acciones:
 - "Valores por defecto" muestra una serie de valores por defecto que se aplicarán a la hora de añadir nuevos valores:
 - "Columnas de tablas de valores" permite seleccionar los elementos de nómina a visualizar en la tabla de valores.
 - "Trabajar con pagas bloqueadas" permite editar valores de elementos de nómina para pagas abiertas y bloqueadas.
 - "Permitir valores retroactivos" permite modificaciones sobre valores cuya fecha inicio es anterior a la fecha de bloqueo
 - "Inserción masiva" permite incrementar o reducir en una cantidad fija o porcentual los valores de los elementos de nómina de tipo monetario, numérico, fecha o cadena que se están viendo en una tabla de valor, así como asignarles un valor fijo. La inserción puede aplicarse a los elementos con valor, sin valor o a todos los que se estén visualizando.

-
- La parte inferior es la zona de edición de valores (propriadamente la tabla de valores). En esta tabla se muestran los valores válidos para las fechas de filtro organizados en formato tabla: en las columnas aparecen los elementos de nómina (máximo 300 registros) y en las filas aparece la población (máximo 100 registros). La tabla de valores aparecerá deshabilitada para pagas cerradas y para pagas abiertas y bloqueadas excepto si se ha seleccionado la opción Trabajar con pagas bloqueadas. Cada una de las celdas tiene un botón Detalles que muestra un formulario donde se puede asignar las fechas de inicio y fin (en caso de no querer utilizar las fechas por defecto), se puede editar el valor y se puede acceder a las pantallas de los históricos de fecha de pago y fecha de imputación. Las celdas con valores retroactivos (fecha de inicio anterior a la fecha de bloqueo) aparecerán deshabilitadas y sólo se podrá modificar la fecha fin con el formulario Detalles excepto si estamos trabajando con la opción Permitir valores retroactivos. La tabla de valor permite, además, acceder al formulario Edición por clave primaria por cada valor de población.

Como se ha dicho anteriormente este tipo de tablas de valores que representan los valores de elementos de nómina a nivel organizativo. Por tanto, para seleccionar la población de trabajo (lugares de trabajo, empresa, etc.) se suele establecer un filtro de nodo como acceso al formulario principal. En el ejemplo de un capítulo posterior se verá con detalle cómo restringir la población.

Insertión masiva de valores fijos

A diferencia de las tablas de valor fijas, en las tablas de valor masivas de valores fijos no se produce el filtrado de periodos del RH o de roles inicial por tratarse siempre de valores nuevos. Será el propio usuario el que indique en la tabla a qué periodos o qué roles asigna el valor para el elemento de nómina que corresponda.

El aspecto general de este formulario es el siguiente:

La principal diferencia entre esta presentación y la de inserción masiva de valores de incidencias es que la cabecera presenta fechas en vez de pagas, y que existe la posibilidad de trabajar sin fecha de fin, marcando la casilla de verificación correspondiente.

La aplicación ofrece la posibilidad de importar valores de forma masiva desde una hoja Excel con la que trabaje normalmente. A partir del archivo de Excel, puede introducir diferentes valores de forma rápida para elementos de nómina y varios periodos del RH o varios roles. Esta importación permitirá cargar las filas y columnas de una hoja Excel a las presentaciones de las tablas de valor de entrada masiva tanto del periodo como del rol.

Valores incidencias

Hay tres diferencias principales entre esta pantalla y la de valores fijos:

- En lugar de tener dos fechas por las que filtrar, aparecen dos pagas, que por defecto se corresponden con la primera paga abierta no bloqueada.
- En incidencias no se filtra a una fecha determinada, sino entre las fechas de inicio y fin de una paga; así que se puede dar el caso en que haya más de un registro en cada celda (cambios de valor a lo largo de la paga). Para insertar, editar y borrar este tipo de registros, se utiliza el formulario Tramos

(descrito más adelante), al que se tiene acceso mediante un botón contiguo al de Detalles. El botón de tramos está deshabilitado si la celda tiene valores retroactivos (fechas inicio anterior a fecha de bloqueo). Si al cargar los valores hay una celda en la que hay más de un registro, aparece sólo el primer registro y cambia la forma del icono (de un lápiz a dos) para indicar que hay más información de la que se puede ver en el formulario.

- Este tipo de tabla de valores es de carácter individual, por tanto, la población o filas serán los periodos RH o los roles RH.

Como ejemplos representativos de este tipo de tablas de valores tenemos **Valores de incidencia por periodo** y **Valores incidencias del rol**.

Visualmente el formulario es muy similar al de valores fijos:

Ahora la zona del filtro presenta pagas en vez de fechas. Las filas de la tabla de valores son de carácter individual no organizativo.

De la misma forma que los valores fijos representan los valores de elementos de nómina a nivel organizativo. Aquí, para restringir la población de trabajo individual (periodos RH, roles, etcétera.) se podrá establecer un filtro de nodo como acceso al formulario principal. En el ejemplo de un capítulo posterior se verá con detalle.

Tramos

A este formulario sólo se puede acceder desde incidencias. La parte superior muestra los datos que vienen de la página de incidencias, en la parte inferior aparece los registros correspondientes y en el centro está el formulario para insertar y editar datos.

Inserción masiva de valores de incidencias

Este formulario permite introducir valores rápidamente para una paga de imputación, una de pago y un elemento de nómina. No se realiza ninguna carga porque el usuario sabe a quién quiere asignar los valores. Cuando los valores se han insertado correctamente, la zona de valores se queda vacía para que el usuario pueda seguir insertando; pero si algún valor es erróneo, se quedan los valores en pantalla marcándose de forma especial los valores erróneos y deshabilitándose los correctos. De esta forma el usuario puede ver cuáles puede corregir.

Como ejemplos representativos de este tipo de tablas tenemos la inserción masiva de valores de incidencias del periodo RH o del rol.

El aspecto visual del formulario principal es el siguiente:

The screenshot shows a software window titled "4 Massive Entry of HR Period Incidents - Standard Company". The window has a menu bar with "File", "Edit", "View", and "Help". Below the menu bar are icons for "Menu", "Print", "Save", "Dates", "Societies", "Reload", and "List". The main area is titled "Massive Entry of HR Period Incidents" and contains several input fields: "Allocation Date" (24/08/2010), "Payment Date" (24/08/2010), "Payroll Item" (empty), "Start Date" (01/08/2010), "End Date" (31/08/2010), "Priority" (Medium Low), and "Change Reason ID" (empty). To the right of these fields is a sidebar with a list of actions: "New Values", "Show / Hide Details", "Work with Blocked Pays", "Allow Retroactive Values", "Import from Excel File", and "Establish Pay Properties". Below the input fields is a table with the following columns: "HR ID", "Period Number", "Name", and "Value". The table has 10 rows, each with a small icon in the first column.

Este tipo de tablas son de carácter individual (periodo RH o rol), pero no tienen filtro de nodo previo de acceso a ellas por tratarse siempre de nuevos valores.

La aplicación incorpora la posibilidad de importar los valores de forma masiva desde una hoja de Excel con la que trabaje normalmente. A partir del archivo de Excel, puede introducir diferentes valores de forma rápida para elementos de nómina y varios periodos del RH o varios roles, así como para una paga. Esta importación permitirá cargar las filas y columnas de una hoja Excel a las presentaciones de las tablas de valor de entrada masiva tanto del periodo como del rol.

Pantallas auxiliares

A continuación se muestran las presentaciones auxiliares descritas en los anteriores apartados.

Valores por defecto

En esta presentación se definen los valores por defecto que se asignarán al insertar registros nuevos. Sólo se aplicarán si el nuevo registro tiene vacíos estos valores en la celda.

The screenshot shows a dialog box titled "Default values - Company 1". It contains the following fields and controls:

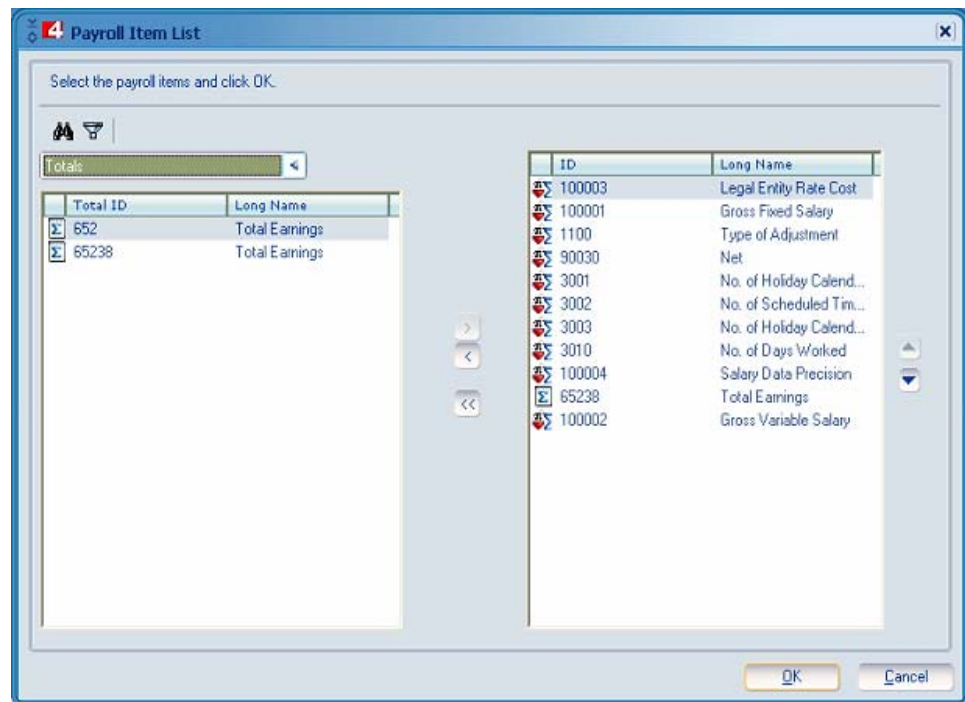
- Start:** A date field with the value "01/06/2007".
- End:** An empty date field with a "No End Date" checkbox checked next to it.
- Priority:** A dropdown menu showing "Medium Low".
- Change Reason ID:** An empty dropdown menu.
- Currency:** A sub-dialog or section containing:
 - Currency ID:** A field with "EUR" and a corresponding "Euro" label.
 - Exchange Date:** A date field with the value "06/06/2007".
- Buttons:** "OK" and "Cancel" buttons at the bottom right.

Esta presentación se llama desde el formulario principal de valores fijo y de valores incidencias.

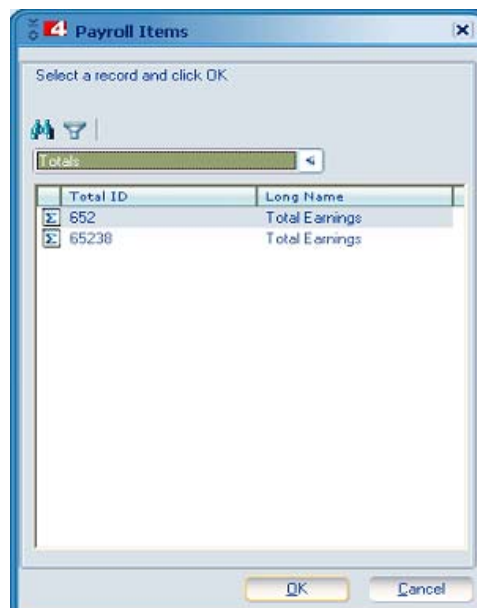
Columnas de tablas de valores

En esta presentación se pueden seleccionar los elementos de nómina sobre los que se quieren asignar valores en la tabla de valores. Los elementos serán los correspondientes al grupo de componentes del periodo o del rol del nivel de cada tabla de valor.

Para las tablas de valores fijos e incidencias el formulario se llama desde Acciones y su aspecto es:



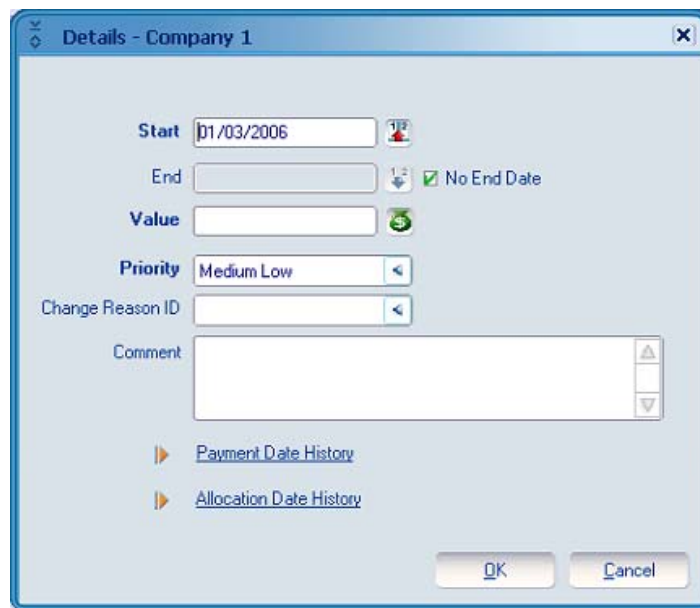
Para tablas de valores de incidencias masivas la plantilla sólo se permite la selección de un elemento de nómina desde la cabecera y su aspecto es:



Detalles

A esta presentación se puede acceder desde tablas de valores fijos e incidencias. Como se ha dicho anteriormente, muestra los detalles del valor asignado a un elemento de nómina.

Su aspecto es el siguiente:



The screenshot shows a window titled "Details - Company 1". Inside, there are several input fields and controls:

- Start:** A date field containing "01/03/2006" with a calendar icon.
- End:** An empty date field with a calendar icon and a checked checkbox labeled "No End Date".
- Value:** An empty text field with a currency icon.
- Priority:** A dropdown menu showing "Medium Low".
- Change Reason ID:** A dropdown menu.
- Comment:** A large text area.
- Payment Date History:** A link with a right-pointing arrow.
- Allocation Date History:** A link with a right-pointing arrow.
- Buttons:** "OK" and "Cancel" buttons at the bottom right.

Histórico fecha de pago e Histórico fecha de imputación

Se accede desde el formulario de Detalles. Muestra la evolución a lo largo del tiempo del valor correspondiente a un elemento de nómina y a un valor de la población. Esta evolución puede ser tanto por pago como por imputación. Sólo se permite la consulta de los valores.

Value History - Company 1

Value History

Allocation filter: 01/03/2005

Legal Entity ID: 0001

Payroll concept: 31030 - Agreed Gross

	Start	End	Value	
--	-------	-----	-------	--

Value History - Company 1

Value History

Allocation Date: 26/06/2006

HR ID - Period No.: M10009 1.00000000

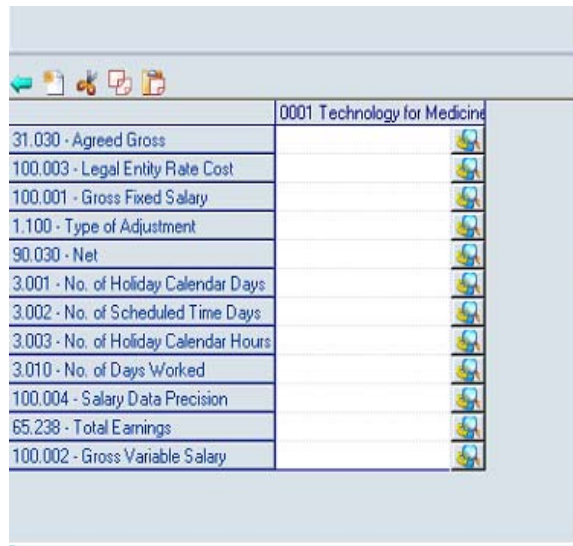
Payroll Item: 1317 - Gross Fixed Salary

	From	Start	End	Value	
--	------	-------	-----	-------	--

Edición por clave primaria, pivotado o de proyección

Se accede desde las tablas de valores fijos e incidencias pulsando sobre un elemento de la población en la columna de la izquierda de la tabla. Muestra los valores de todos los conceptos correspondientes a un ejemplar de población. Permite insertar, modificar y borrar.

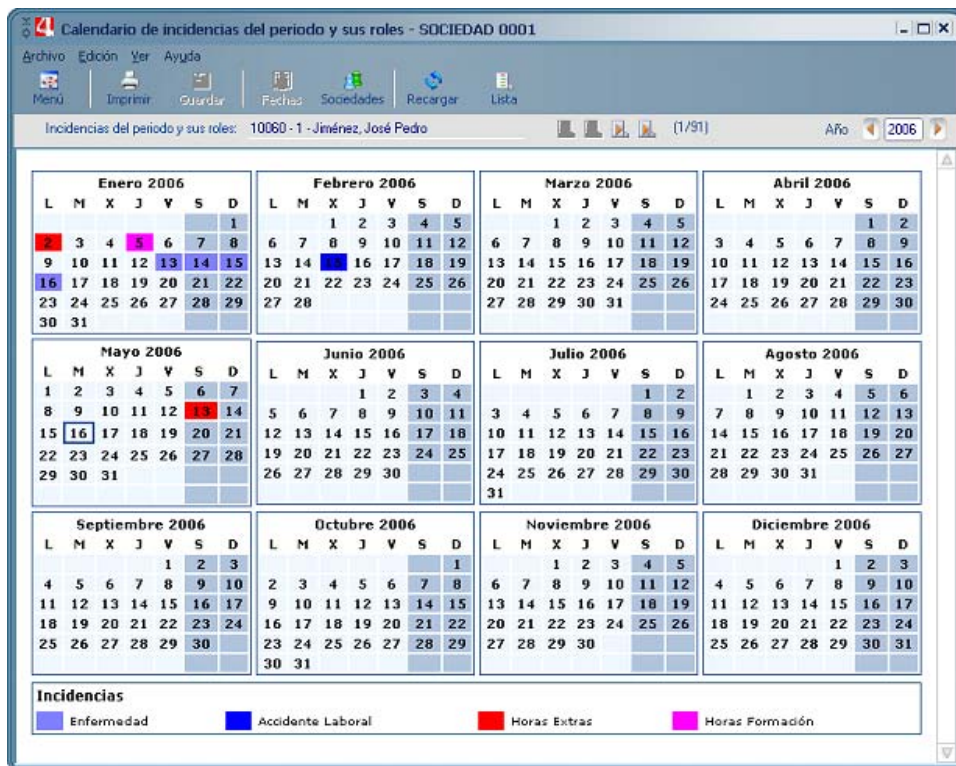
Su visualización se realiza sobre la misma tabla de valores:



	0001 Technology for Medicine
31.030 - Agreed Gross	
100.003 - Legal Entity Rate Cost	
100.001 - Gross Fixed Salary	
1.100 - Type of Adjustment	
90.030 - Net	
3.001 - No. of Holiday Calendar Days	
3.002 - No. of Scheduled Time Days	
3.003 - No. of Holiday Calendar Hours	
3.010 - No. of Days Worked	
100.004 - Salary Data Precision	
65.238 - Total Earnings	
100.002 - Gross Variable Salary	

Plantilla de calendario

La plantilla de calendario tiene como objetivo permitir mostrar información en formato calendario en una presentación. Por ejemplo, para mostrar el calendario de incidencias del periodo y sus roles en el último año (SRCO_TA_INC_CAL).



A través de esta plantilla puede configurar el aspecto de la presentación haciendo uso de distintos parámetros, por ejemplo, mes y año de inicio del calendario, número de meses por fila, título de la página y de la leyenda.

Los colores usados se obtienen rotando una lista que contiene 15 colores asociados por orden al parámetro "ListIDs". La lista de colores se corresponde con los siguientes códigos a nivel gráfico:

'#8080FF', '#0000FF', '#FF0000', '#FF00FF', '#FFFF00', '#00FFFF', '#FF0080', '#00FF80', '#80FF00', '#8000FF', '#FFFF80', '#80FFFF', '#808040', '#804080', '#408080':

En caso de periodos solapados, un periodo sobrescribe el periodo anterior. Es decir, el último periodo (parámetro "PeriodsToDraw") es el que domina.

A continuación se detallan los parámetros asociados a esta plantilla:

- StartMonth: Mes de inicio del calendario (número : 1 - 12)
- StartYear : Año de inicio del calendario (número)
- ListIDs: Lista de posibles identificadores (Cadena). Ejemplo: "'001', '002', '003', '004', '005'"

- ListNames: Lista de nombres asociados a los identificadores (cadena). Ejemplo: "'Vacations','Sickness','Maternity Leave','Paternity Leave','Unpaid Leave'
- PageHeader: Cabecera de la página (cadena)
- LegendTitle: Título de la leyenda (cadena)
- IDsToDraw: Lista de identificadores a dibujar (cadena). Ejemplo: "'001','002'"
- PeriodsToDraw: Lista de los periodos asociados a los identificadores (IDsToDraw) (cadena). Ejemplo: "'20-08-2005', '22-08-2005', '23-08-2005', '23-08-2005'"
- Today: Fecha a marcar con un borde. Ejemplo: '20-08-2005'
- NumberOfMonths: Número de meses a mostrar en el calendario (número: 1 - n). Valor por defecto: 1
- NumberMonthsPerRow: Número de meses a mostrar por fila (número: 1 - n). Valor por defecto: 1
- NumberColumnsLegend: Número de columnas en la leyenda (número: 1 - n). Valor por defecto: 2
- HighlightDays: Días a marcar en el calendario, normalmente el fin de semana (cadena). Ejemplo: "'5','6'"
- DateFormat: Formato de la fecha. Ejemplo: "yyyy-MM-dd". Formato por defecto: "yyyy-MM-dd"
- DateSeparator: Separador de fechas. Ejemplo: "-". Valor por defecto: "-"

Aquellos parámetros que contengan un valor por defecto no es necesario que contengan información, es decir, pueden ir en blanco.

Plantilla de desktop

Esta plantilla tiene como objetivo la creación de presentaciones donde se tengan accesibles los recursos de trabajo más habituales: menús, gráficos, informes, poblaciones, procesos de negocio y lista de tareas.

Al igual que el resto de las plantillas esta formada por un objeto presentación y un objeto Meta4Object, pero además requiere de una configuración específica donde se indicará para cada desktop el contenido de los diferentes recursos.

Por ejemplo, para el módulo de evaluación se puede tener un desktop que reúna las opciones de menú, procesos de negocio, informes y gráficos relacionados con el módulo.

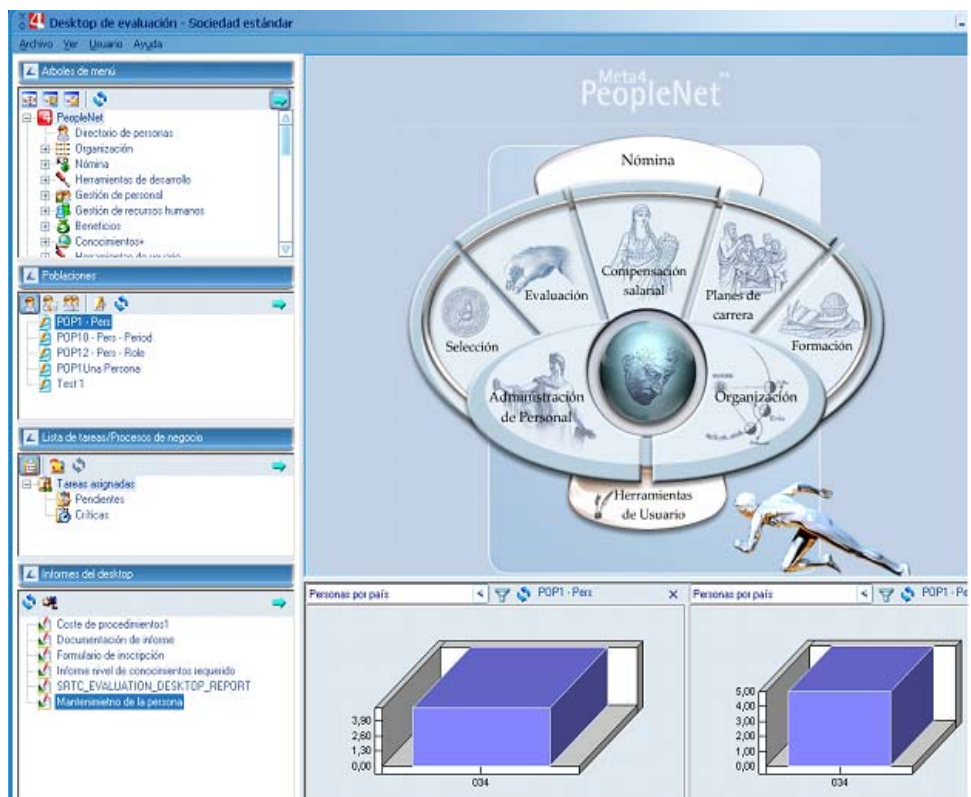
Existen dos plantilla de desktops:

- Plantilla tecnológica: Plantilla base de desktops accesible siempre.
- Plantilla global: Plantilla que partiendo de la plantilla tecnológica viene enriquecida con los aspectos globales de desktops, principalmente los relacionados con la funcionalidad de poblaciones.

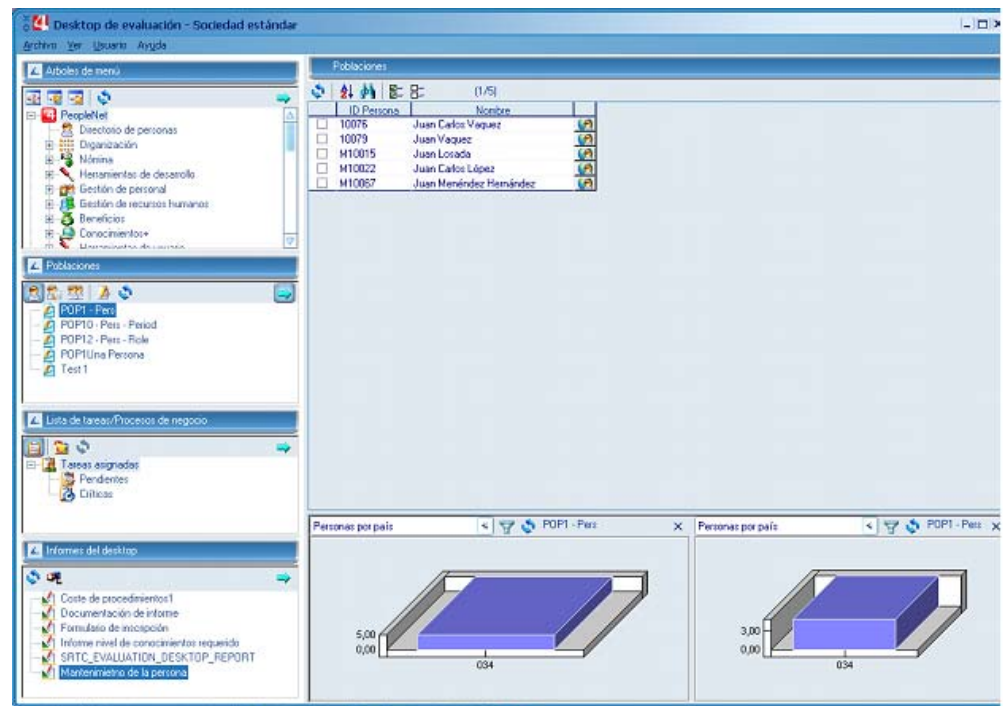
A la hora de desarrollar un nuevo desktop hay decidir de qué plantilla se parte, y esto dependerá del producto que se tenga instalado. Si el producto que se tiene instalado dispone de la funcionalidad de poblaciones y de la plantilla global de desktops (Meta4Object y presentación: SRCO_GN_DESKTOP) se trabajará con la plantilla global de desktops en otro caso se utilizará la plantilla tecnológica de desktops.

El aspecto de un desktop siguiendo la plantilla global varía ligeramente en función del recurso que se seleccione a visualizar en la zona izquierda. A continuación se muestran las distintas vistas:

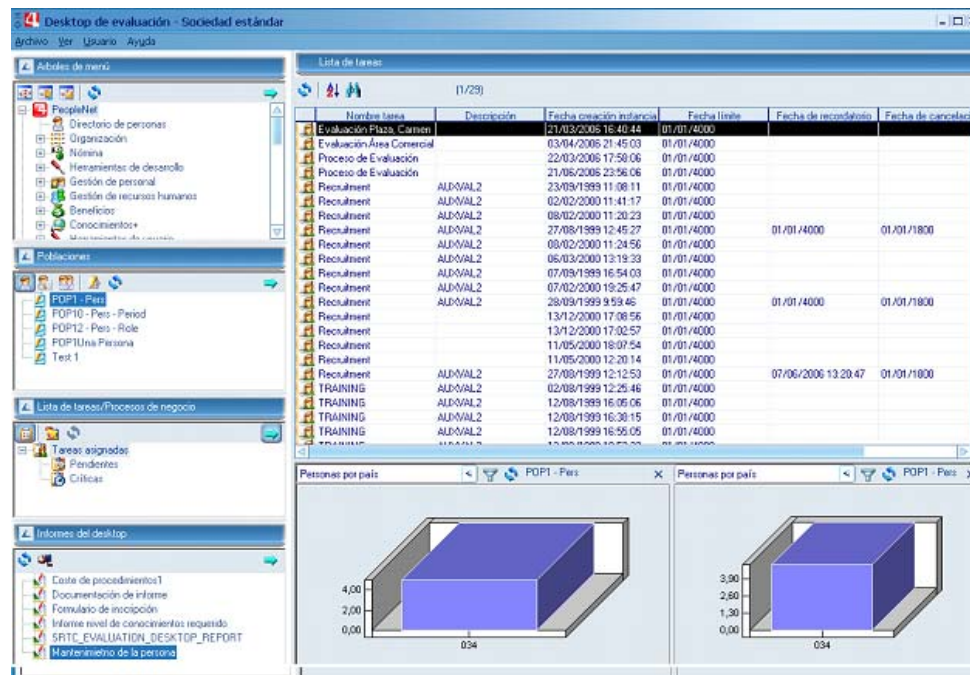
Aspecto de un desktop con la vista de menús seleccionada:



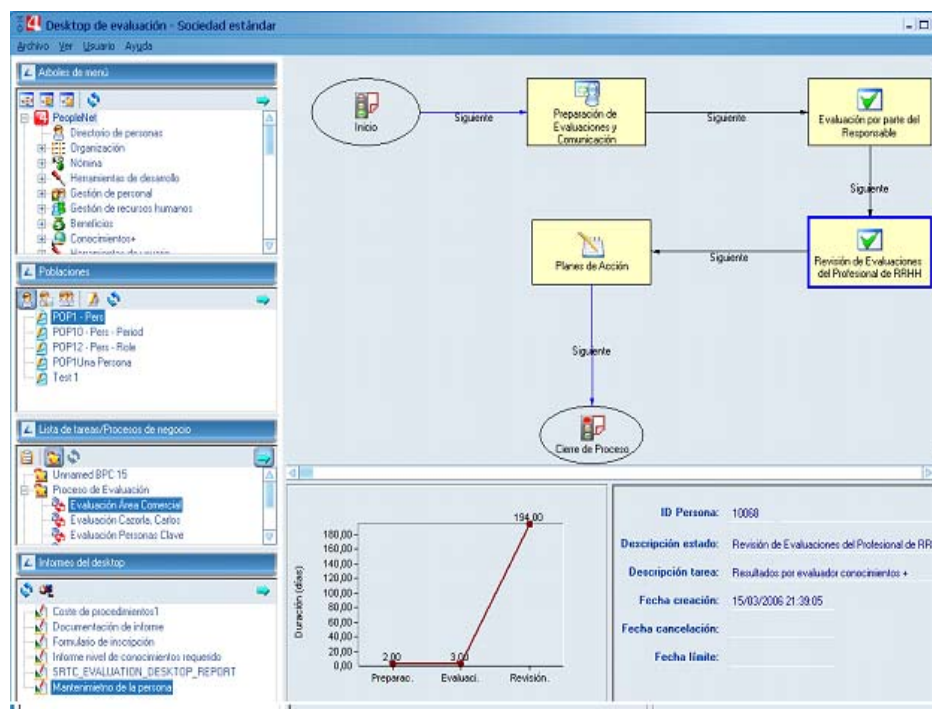
Aspecto de un desktop con la vista de poblaciones seleccionada



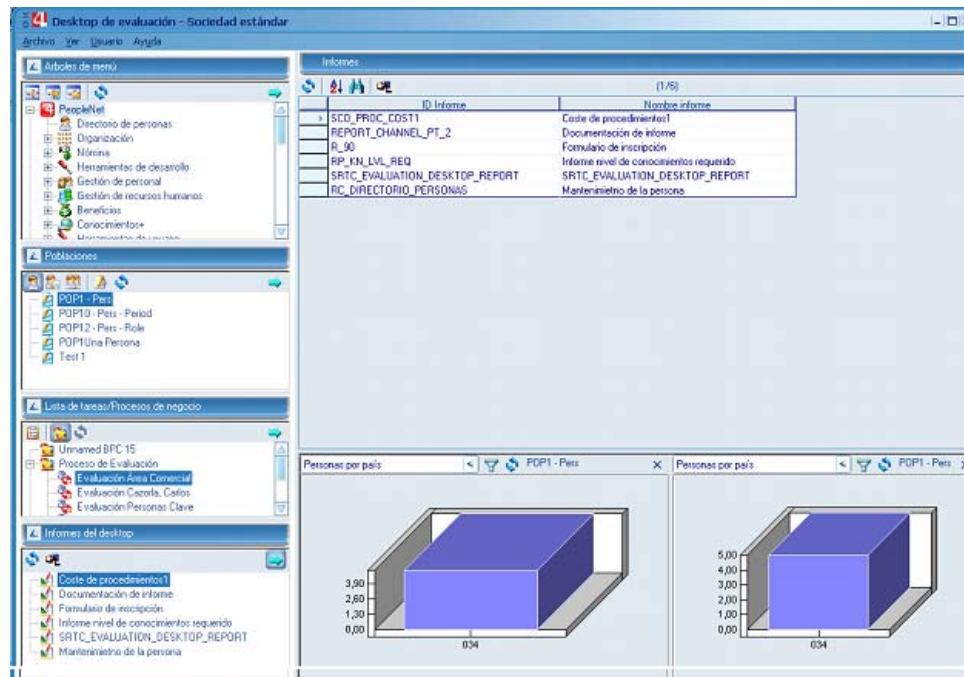
Aspecto de un desktop con la vista de lista de tareas seleccionada



Aspecto de un desktop con la vista de procesos de negocio seleccionada



Aspecto de un desktop con la vista de informes seleccionada



En general las distintas vistas se pueden dividir en tres zonas bien diferenciadas:

- La parte izquierda presenta el conjunto de recursos disponibles:
 - Menús. Este recurso tiene tres vistas posibles, menú completo, menú de favoritos y menús asociados al desktop. Además permite gestionar los menús favoritos añadiendo/borrando opciones de menú e incluso organizándolas en carpetas.
 - Poblaciones. Este recurso tiene tres vistas posibles: persona, periodo y rol, de forma que en cada vista se muestran las poblaciones válidas para cada nivel. Además permite acceder al editor de poblaciones.
 - Listas de tareas/ Procesos de negocio. Este recurso muestra una clasificación de la lista de tareas asignadas: pendientes y críticas, y también permite visualizar el conjunto de procesos de negocio que se hayan configurado para el desktop así como las instancias no completadas de los mismos.
 - Informes. Este recurso permite ver el conjunto de informes y tareas de lanzamiento de informes que se haya configurado para el desktop así como la ejecución de los mismos.

- La parte superior derecha presenta información ampliada del recurso seleccionado en la parte izquierda. La forma de seleccionar el recurso es mediante la flecha que aparece en la parte superior derecha de cada recurso:
 - Menús. Muestra la página html relacionada con el menú seleccionado en la izquierda.
 - Poblaciones. Se muestra un listado con los elementos que forman la población seleccionada en la parte izquierda. La lista tiene formato multiselección pues se puede aplicar como filtro en los gráficos y ejecución de tareas que trabajan con filtrado de poblaciones (por ejemplo cuando se ejecuta la opción de menú de **Mantenimiento de la persona**).
 - Listas de tareas/ Procesos de negocio. Cuando se está en la vista de lista de tareas se muestra el conjunto de tareas asignadas al usuario logado en la aplicación bien sea todas, sólo las no críticas o las críticas en función del tipo de tarea seleccionado en el recurso de la izquierda, además permite la ejecución de las mismas. Cuando se está en la vista de procesos de negocio permite ver el diagrama de estados asociado al proceso de negocio seleccionado, además si se está situado en una instancia se marca en el estado en el que se encuentra la misma.
 - Informes. Muestra en formato lista el conjunto de informes y tareas de lanzamiento de informes que se haya configurado para el desktop y permite la ejecución de los mismos.
- La parte inferior derecha muestra los gráficos asociados al desktop, excepto cuando se ha seleccionado en el recurso de la izquierda la vista de procesos de negocio, en cuyo caso se muestra un gráfico de la evolución de la instancia del proceso de negocio seleccionada (Conjunto de estados por los que pasa y el su duración en días), junto con una ficha de información de la instancia.

Exportación/Importación desde Excel

PeopleNet ofrece la posibilidad de importar/exportar datos de nuevas altas, empleados etc, usando una plantilla Excel previamente configurada para usted. De esta forma solo es necesario establecer en esta herramienta los parámetros de importación/exportación según los datos requeridos

Dichos parámetros de importación/exportación son:

Criterios generales para realizar el proceso

Datos requeridos si se va a llevar a cabo una importación o una exportación tales como ID del proceso, Nombre de proceso y Comentario.

Dependiendo si elige importar o exportar las opciones pueden variar:

Gestionar errores (solo para importación): Genera una hoja resultado de la importación indicando los registros erróneos y el texto del error.

Nombre de la plantilla (solo exportación): Plantilla en la que se van a volcar los datos de la exportación. Es necesario el nombre completo, incluyendo la extensión.

Nombre del archivo a generar (solo exportación): Prefijo del archivo sugerido al usuario. Este nombre se completará en el momento de la ejecución del proceso.

Características del proceso

Define las características específicas del proceso de importación/exportación que va a realizar.

Como primer paso se definen ID Característica de proceso, ID Método de carga especial, Método de grabación especial (sólo para importaciones), ID Hoja Excel, **Tipos de listas** (solo exportación), etc

Ejecución de procesos de importación y exportación

Cuenta dos opciones, una para importación y otra para exportación. Una vez se ha seleccionado el proceso que se desea ejecutar:

- **En caso de importación:** Debe localizar el archivo donde se encuentran los datos que quiere importar. La aplicación le sugerirá un nombre para el archivo que se va a generar con los errores que el usuario puede luego modificar. Una vez hecho esto puede ejecutar la importación.
- **En caso de exportación de listas:** La aplicación le sugerirá un nombre para el archivo (basado en la información de las tablas de configuración) que generará con los datos de la exportación. Luego se muestra el listado de los Meta4Objects para los cuales (como se indicó en las tablas de configuración) se quiere un filtro. Indique el filtro deseado. Dependiendo de la configuración accederá al editor de sentencias o a un listado de poblaciones. Una vez hecho esto puede ejecutar la exportación.

-
- **En caso de exportación de multinodos:** El usuario indica el archivo donde están los datos de la clave primaria necesarios para hacer la exportación. La aplicación le sugerirá un nombre para el archivo que se generará con los datos de la exportación. Una vez hecho esto puede ejecutar la exportación.

Metodología

La metodología del framework consiste en una serie de reglas y prácticas de desarrollo recomendable para crear y modificar componentes en el producto. Esta información permitirá a los desarrolladores tomar las decisiones adecuadas a la hora de acometer sus desarrollos. Se trata a continuación:

- Nomenclatura
- Reglas generales
- Traducciones

Nomenclatura

El framework define una nomenclatura que es utilizada en los desarrollos estándar de los productos Meta4 y en las herramientas de desarrollo integradas en el development framework, y que debe ser seguida en el resto de desarrollos de diferentes funcionalidades, principalmente en los que haya que realizar cambios manuales.

En particular se describe la nomenclatura a seguir para los ID de Meta4Objects, estructuras de nodo y presentaciones. También se sugieren recomendaciones para los métodos.

Meta4Objects, estructuras de nodo y presentaciones

Los identificadores de estos tipos de objetos deberán tener la siguiente estructura:

PREFIX_MOD_CLASS_NAME

Donde:

- **PREFIX** es un prefijo que se obtiene del parámetro de aplicación LOGON/SCH_SESSION/PREFIX_OVERWRITE. Normalmente sus valores son:
 - SRTC: Para las plantillas tecnológicas y Meta4Objects de tecnología
 - SRCO: Corporativo
 - SRSP: España
 - SRFR: Francia
 - SRAR: Argentina

Donde:

- S = Estándar
- R =cliente Rich

Si el Meta4Object se hereda en cliente, el nuevo Meta4Object hijo deberá empezar por C (Client), por ejemplo: CRSP: España, CRFR: Francia, etcétera.

- **MOD** es el módulo funcional al que pertenece el objeto, por ejemplo:
 - PA: gestión de personal
 - OR: organización
 - TK: tecnología
 - Etcétera.



El asistente del development framework lo lee desde parámetro de aplicación CHANNEL_DESIGNER/CONFIGURATION/ID_MODULO_FUNC cuando no vienen valores es el valor es TK por defecto.

- **CLASS** es el tipo de objeto o plantilla:
 - "QBF: para nodos y estructuras de nodo que representan filtros QBF.
 - MT: para listas y mantenimientos.
 - IN: para formularios de información.
 - MN: para multinodos.
 - PW: para asistentes de petición de parámetros y ejecución de procesos.
 - VT: para tablas de valores.
- **NAME** es el nombre que debe representar información de la funcionalidad concreta a la que pertenece el objeto.

De esta forma el ID de un Meta4Object de lista y mantenimiento de personas que pertenece al módulo de Gestión de personal quedaría de la siguiente manera: SRCO_PA_MT_PERSON.

Métodos

En el caso de identificadores de métodos y conceptos, siempre es recomendable indicar cuales de ellos pertenecen a la funcionalidad exportada (conocido como API o application program interface) y cuales no, por lo tanto para identificar elementos que no pertenecen al API del Meta4Object, se les añadirá un prefijo PVT_ indicando que son métodos privados.

Reglas generales

Desarrollo - Ciclo de vida

La gestión del ciclo de vida es la administración de los cambios de funcionalidad que se realiza entre versiones.

Creación

Los objetos que son necesarios crear para implementar una nueva funcionalidad se deben intentar crear con las herramientas del framework (mantenimientos, listas, multinodos y formularios de información). Es una creación normal de nuevos objetos.

Localización de objetos estándar

Cuando una funcionalidad estándar se modifica funcionalmente se debe crear un nuevo objeto heredando del objeto estándar para garantizar el mantenimiento en la medida de lo posible. El nuevo objeto se conoce como localizado pues se va a modificar de forma local.

Actualmente las herramientas del development framework soportan creación de objetos locales para mantenimientos, listas, multinodos y formularios de información. La modificación de funcionalidad siguiendo la misma metodología, heredando el Meta4Object y la presentación cuando sea requerido por el desarrollador de funcionalidad.

Modificación

Las modificaciones siempre se deben realizar sobre los objetos de los que somos propietarios, por lo tanto, si el objeto que se quiere modificar es un objeto estándar primero hay que localizar la funcionalidad para crear un objeto de nuestra propiedad y después podremos modificar este objeto (ya de nuestra propiedad). Con las herramientas del framework la localización y la primera modificación del objeto estándar se pueden hacer a la vez.

En general, se habla de modificar algo siempre que se tenga que cambiar el comportamiento funcional de un objeto (realizando la acción de añadir, modificar o borrar, en el caso en que se pueda, en cualquiera de los elementos que forman el objeto). No se entrará en el detalle de si ese objeto está o no localizado, si no lo está, se presupone que antes de modificar habrá que localizarlo.

En algún caso, en el producto estándar se entrega el objeto ya localizado para cliente, para que se modifique directamente; se reconoce por la nomenclatura: CRSP...

Ejecución

Árbol de ejecución y mapeos

Los árboles de ejecución se utilizan para gestionar los objetos que se ejecutan para cada sociedad definida en la aplicación.

A diferencia de los proyectos de desarrollo, se pueden asociar a estos una sociedad. De este modo, los proyectos organizan los objetos a los cuales las sociedades pueden tener acceso.

Los árboles de ejecución y los mapeos permiten definir la ejecución de objetos en un entorno.

Los árboles de ejecución se configuran con la herramienta Explorador de proyectos. Permite definir objetos que se ejecutan para cada sociedad definida en un entorno de ejecución. En concreto el objeto que permitirá definir la ejecución es el mapeo.

Los mapeos permiten reemplazar virtualmente los Meta4Objects y presentaciones por otros siguiendo una parametrización. Por ejemplo:

Para una Organización O1 se debería modificar el Meta4Object A. Si no se desea cambiar todas las referencias (formulario, tarea, etcétera...) a dicho Meta4Object, se puede implementar toda la nueva funcionalidad en un Meta4Object B, y posteriormente mapear ambos.

El mapeo se define para una organización. En el caso de que no existiese ningún mapeo, se utiliza el Meta4Object original.

El mapeo de presentaciones funciona exactamente de la misma forma, según el contexto de organización, se pueden ejecutar las presentaciones mapeadas.

Los editores siempre gestionan el objeto original tanto para Meta4Objects como para presentaciones.

Parámetro de aplicación estándar

El parámetro de aplicación THINCLIENT/PORTAL_PARAM/STANDAR nos permite hacer desarrollos distintos en función de distintas legalidades. En base a este parámetro se construyen los desarrollos globales (nombre y dirección).

Barra de tareas del desarrollador

Al ejecutar la opción Instancias de la herramienta Barra de tareas de desarrollador se podrá ver por cada instancia de Meta4Object:

- Nombre lógico: ID del Meta4Object que se ha pedido ejecutar (Id en la tarea, en la definición de nivel 2, en el código JIT...).

- Nombre real: ID del Meta4Object que se ejecuta en esa sociedad teniendo en cuenta los mapeos.

Cambio de sociedad en las presentaciones

Sólo se permite el cambio a las sociedades donde ese ID Real (el del Meta4Object y el de la presentación) tenga sentido.

Traducciones

Cuando se visualizan datos, es norma general, mostrarlos con un nombre o literal, de forma que el código identificativo resulte transparente al usuario. Así, la traducción de campos, se hará a través de joins con tablas de tipo Look Up como, por ejemplo, el uso de la tabla de sexos en la presentación de personas.

En aquellos casos en los que el elemento a traducir sea resultado de un cálculo o bien éste no provenga directamente de un campo de una tabla, es necesario hacer un método para obtener la traducción, por metodología estos elementos traducidos quedan guardados como mensajes de error.

EJEMPLO

Dado un elemento ITEM_COLOR, se quiere un valor traducido sobre el elemento TRANSLATED_ITEM_COLOR. Habría que definir los mensajes de error necesarios (en este caso 3 mensajes) y utilizar el siguiente código después del cálculo del elemento.

```
sGreen = GetErrorString(125,125,125)
sBlue = GetErrorString(125,125,126)
sRed = GetErrorString(125,125,127)

For i= 0 to (Count()-1)
  Moveto(i)
  If ITEM_COLOR = 0 Then
    TRANSLATED_ITEM_COLOR = sGreen
  ElseIf ITEM_COLOR = 1 Then
    TRANSLATED_ITEM_COLOR = sBlue
  ElseIf ITEM_COLOR = 2 Then
    TRANSLATED_ITEM_COLOR = sRed
  Else
    'By Default
    TRANSLATED_ITEM_COLOR = sGreen
  EndIf
Next
```



Caso Práctico

2

Acerca de este capítulo

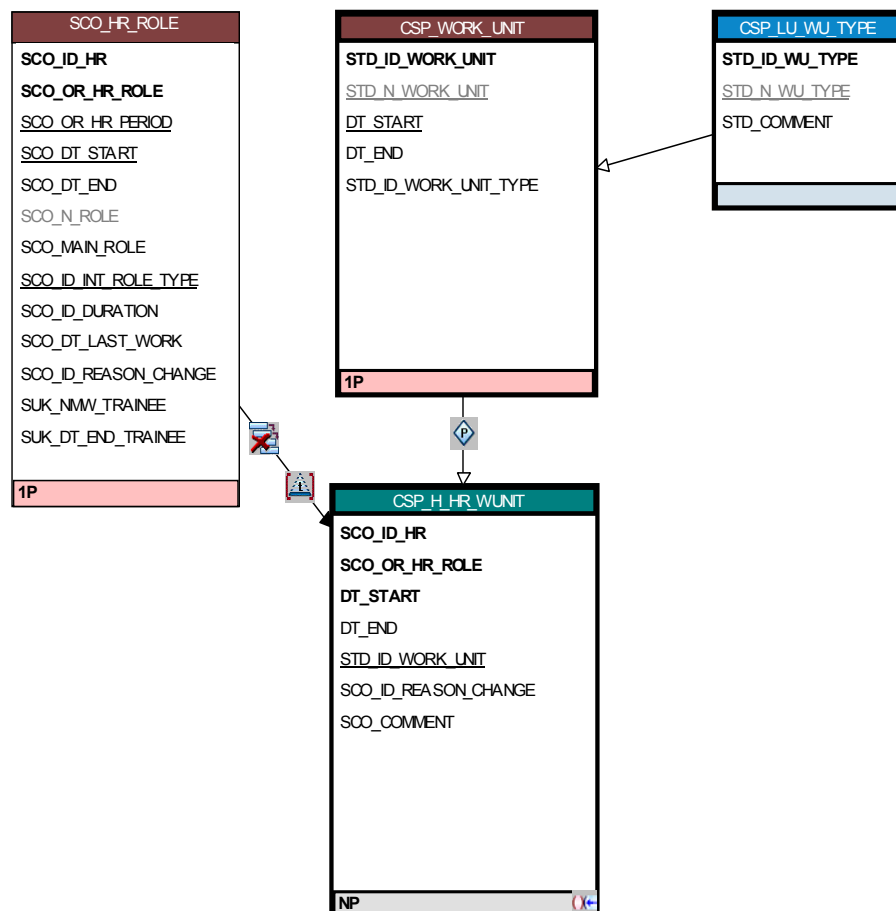
En este capítulo seguiremos la metodología del framework para hacer ejemplos de creación y modificación de los componentes básicos basados en las plantillas definidas anteriormente.

Los ejemplos que se van a mostrar a continuación no tienen por qué corresponderse con funcionalidades reales, pretenden ser simplemente ejemplos técnicos.

La nomenclatura que se utilizará si los objetos no existen será CEX o CSP

Los objetos relativos al modelo de datos se suponen creados.

Creación de una lista y mantenimiento



Se crea una lista y un mantenimiento a partir de la tabla básica CSP_LU_WU_TYPE (Tipo unidad org.) que contiene tres campos, el ID, el nombre y un campo de comentario

El Meta4Object que se crea se utiliza tanto para el listado de datos como para el mantenimiento de datos.

La presentación por norma general se crea sólo para el mantenimiento ya que la de la lista no es necesaria ya que se genera en ejecución.

CSP LU WU TYPE
STD_ID_WU_TYPE
STD_N WU TYPE
STD_COMMENT

Para la creación tanto de la lista como del mantenimiento seleccione la opción:

Herramientas de desarrollo | Desarrollar con el development framework | Crear lista y mantenimiento

A continuación, se detallan los diferentes pasos a seguir, teniendo en cuenta las diferentes fases y pasos del wizard.

Fase 1 (1 de 5)

Indicar la tabla sobre la que se va a construir el Meta4Object.

Fase 1 (2 de 5)

Indicar el identificador del nuevo Meta4Object. En nuestro caso CRSP_OR_MT_LU_WU_TYPE donde:

- CRSP: Cliente Rich
- OR: Módulo de organización
- MT: Objeto de lista y mantenimiento
- LU_WU_TYPE: funcionalidad del objeto.

La casilla *Cacheable* está marcada porque la tabla se ha definido así (cacheable de datos), pero esto se puede modificar si se desea.

El ID nodo e ID estructura de nodo serán los mismos que el del Meta4Object.

Revisar que en la sentencia están todos los campos de la tabla con los que vamos a querer trabajar tanto en la lista como en el mantenimiento, en nuestro caso todos.

Fase 1 (3 de 5)

Seleccionar los campos a mostrar en la lista. En nuestro caso el ID y el nombre.

Fase 1 (5 de 5)

En el ejemplo el paso Fase 2 (4 de 5) no aparece porque se ha indicado que no se quiere QBF.

Para grabar la información relativa al Meta4Object, generar el mantenimiento, y editar la presentación seleccione la opción **Grabar y editar presentación**.

Edición de la presentación

Sólo para los casos en los que haya seleccionado la opción Grabar y editar presentación en la Fase 1(5/5).

Paso 1 de 3

Revisar que el ID de presentación es el deseado (en nuestro caso el mismo que el Meta4Object)

Paso 2 de 3

No añadimos ningún enlace.

Paso 3 de 3

En la presentación, en la tabla de la parte inferior se muestran los campos ID y nombre (en el formulario se ven todos).

Grabar la presentación.

Ejecutar la presentación para ver que se muestra lo deseado.

Al finalizar tenemos:

- Un Meta4Object (CRSP_OR_MT_LU_WU_TYPE) que utilizamos tanto para listar como para mantener los datos.
- Una presentación (CRSP_OR_MT_LU_WU_TYPE que utilizamos para mantener los datos.

Siguiendo este mismo ejemplo, crear el meta4object y presentación de mantenimiento y lista para la tabla CSP_WORK_UNIT.

Al finalizar tenemos:

- Un Meta4Object (CRSP_OR_MT_WORK_UNIT) que utilizamos tanto para listar como para mantener los datos . En este caso la casilla Cacheable no está marcada porque la tabla no se ha definido así (cacheable de datos). Lo

que nos interesa es una lista con filtro QBF, y en la fase Fase 1 (4 de 5) se detallan los campos a mostrar en el filtro. En nuestro ejemplo se muestran el id, el nombre y el tipo de departamento

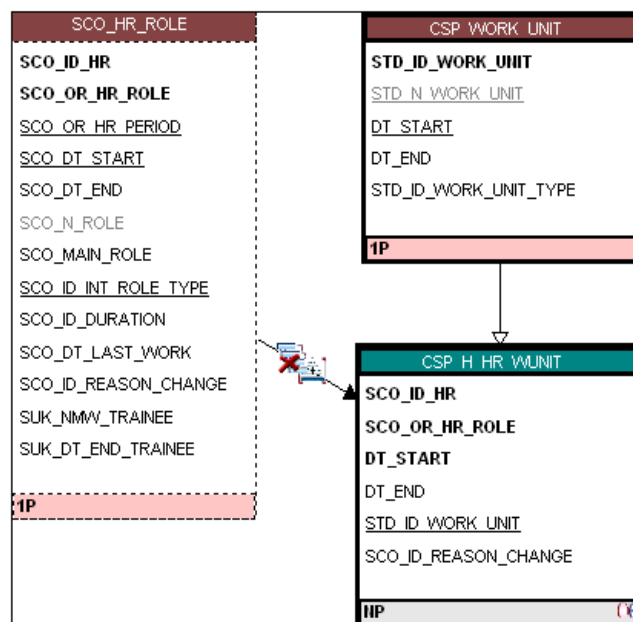
- Una presentación (CRSP_OR_MT_WORK_UNIT) que utilizamos para mantener los datos. Opcionalmente crearemos la presentación de filtro QBF.CRSP_OR_MT_WORK_UNIT.

Creación de un multinodo

Se crea un multinodo a partir de dos tablas:

- SCO_HR_ROLE (Roles).
- CSP_H_HR_WUNIT (Unidad org. RH) que contiene los campos ID y Ordinal del role, ID del departamento, fecha de inicio y fecha de fin en el departamento, ID de razón de cambio al departamento y un campo de comentarios.

Y que tienen una relación con la tabla del ejemplo anterior



Como es necesaria una lista como entrada al multinodo, se crea una siguiendo los pasos del ejemplo anterior para el objeto CSP_LU_WU_TYPE y que se llame CRSP_PA_MT_HR_ROLE o se reutiliza una ya existente en el producto

estándar.

En este caso la lista sí tiene QBF y en la fase Fase 1 (4 de 5) se detallan los campos a mostrar en el filtro (para el ejemplo); mostramos el id y el nombre.

Para esta lista no se genera presentación ya que el mantenimiento de los datos de esta tabla sólo se hace a través del multinodo que se crea a continuación; por lo tanto en el punto Fase 1 (5 de 5) hay que seleccionar la opción **Grabar**.

Para la creación del multinodo seleccione la opción:

Herramientas de desarrollo | Desarrollar con el development framework | Crear multinodo

A continuación, se detallan los diferentes pasos a seguir, teniendo en cuenta las diferentes fases y pasos del wizard.

Fase 1 (1 de 4)

Indicar la tabla del nodo raíz que en nuestro caso es SCO_HR_ROLE

Fase 1 (2 de 4)

- Indicar el identificador del nuevo Meta4Object. En nuestro caso CRSP_OR_MN_HHR_WUNIT donde:
- CRSP: Cliente Rich
- OR: Módulo de organización
- MN: Multinodo.
- HHR_WUNIT: funcionalidad del objeto.

Revisar que en la sentencia están todos los campos de la tabla necesarios, en el ejemplo todos.

Fase 1 (3 de 4)

Posicionarse en el nodo raíz.

Como se quiere que el Meta4Object tenga una lista de entrada, haga clic en el botón Listas y seleccione la lista creada anteriormente llamada CRSP_PA_MT_HR_ROLE.

Como la tabla de la lista y del nodo raíz es la misma, hay que definir:

- Tipo de relación: 1:1

-
- Nodo: CRSP_PA_MT_HR_ROLE
 - Elementos de la lista: SCO_ID_HR-SCO_ID_HR;
 - SCO_OR_HR_ROLE-SCO_OR_HR_ROLE.

Definir el nodo hijo del multinodo haciendo clic en **Nuevo**.

Fase 2 (1 de 1)

Indicar el nombre del nuevo nodo: CRSP_OR_MN_HHR_WUNIT

Seleccionar del listado la tabla CSP_H_HR_WUNIT

Seleccionar todos los campos y por ser una tabla temporal se ordena por fecha de inicio descendente (para tener acceso al registro más reciente)

Se crean automáticamente los conectores entre los campos relacionados de las dos tablas (la del nodo raíz y la del nodo hijo) si están relacionadas por modelo de datos. En el ejemplo no es necesario entrar en la definición de conectores porque están definidos en el modelo de datos.

Edición de la presentación

Sólo para los casos en los que haya seleccionado la opción Grabar y editar presentación

Fase 1 (4 de 4)

Para crear la presentación asociada, seleccionar Grabar y editar presentación.

Fase 1 (1 de 2)

Indicar el ID de la presentación, en el ejemplo el mismo del Meta4Object CRSP_OR_MN_HHR_WUNIT.

Fase 1 (2 de 2)

Se marcan todos los nodos porque se desea visualizar todos en la presentación.

Posicionarse en el nodo raíz y hacer clic en Editar:

- **Fase 2 (1 de 2):** no se requieren enlaces asociados a este nodo
- **Fase 2 (2 de 2):**
 - en la presentación, en la tabla de la parte inferior se muestran los campos ID y nombre (en el formulario se ven todos).

- posicionarse en el nodo hijo y hacer clic en Editar:
 - **Fase 2 (1 de 2):** No se requieren enlaces asociados a este nodo
 - **Fase 2 (2 de 2):** En el formato tabla queremos el ID la fecha de inicio, la de fin y el valor

Creación de un formulario de información

No hay herramienta en el framework para la creación de un Meta4Object de información, pero los Meta4Objects multinodo son reutilizables como Meta4Objects de información.

La funcionalidad de filtro de fechas (que es la única que distingue el multinodo de la información) está habilitada en los Meta4Objects multinodo creados con las herramientas.

Si en el formulario de información se muestra la misma información que en el multinodo se puede reutilizar el mismo Meta4Object sin influir en la funcionalidad. Sería suficiente con crear una nueva presentación sobre este mismo Meta4Object. En este caso, al reutilizar el Meta4Object para las dos funcionalidades debería haberse llamado CRSP_OR_MNIN_HHR_WUNIT.

Creación de la presentación

Para la creación de la presentación de formulario de información se utilizan las herramientas del framework:

Herramientas de desarrollo | Desarrollar con el development framework | Crear formulario de información.

Fase 1 (1 de 3)

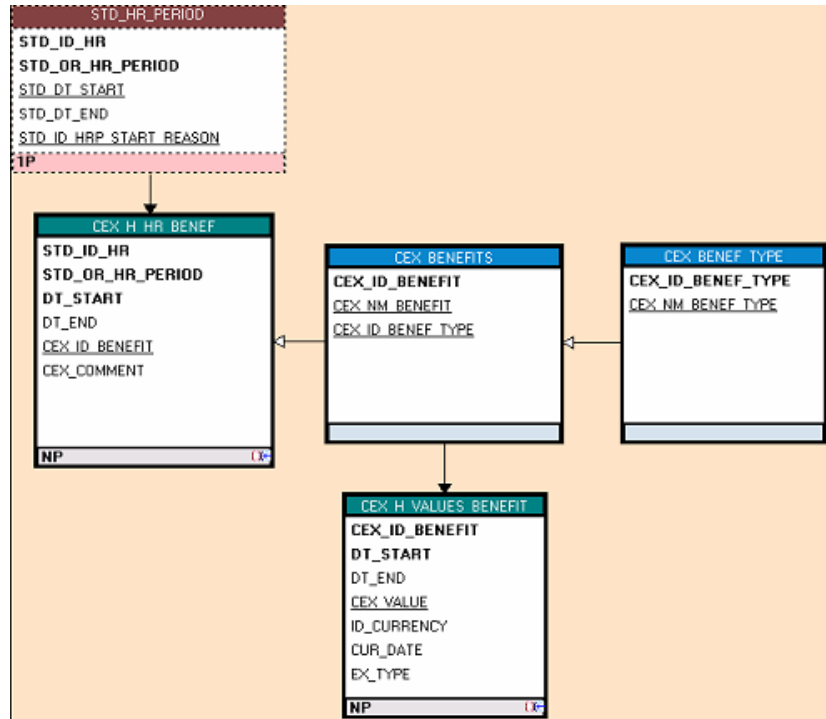
La nueva presentación se llama CRSP_OR_MNIN_HHR_WUNIT

Fase 2 (2 de 3)

Posicionarse en cada nodo y para cada uno de ellos, indicar:

- Si se quiere visualizar en formato formulario o en formato tabla.
- Si se quiere acceder a algún enlace

En el siguiente ejemplo, módulo de beneficios del empleado, se quiere añadir otra información al formulario de información que no está en el multinodo y esta información es el listado de empleados que tiene ese beneficio, es la información de la tabla CEX_H_HR_BENEF.



Por lo tanto, la forma de crear un formulario de información sería utilizar la herramienta de multinodos para la creación del Meta4Object y generar la presentación con la herramienta del framework de presentaciones de información.

Se crea un formulario de información con datos de 3 tablas CEX_BENEFITS, CEX_H_VALUES_BENEFIT y CEX_H_HR_BENEF.

Para la creación del Meta4Object de formulario de información se utiliza las herramientas del framework.

Herramientas de desarrollo | Desarrollar con el development framework | Crear multinodo

A continuación, se detallan los diferentes pasos a seguir:

- Creación del Meta4Object aprovechando herramienta multinodo
- Una vez diseñado el Meta4Object multinodo se realizan unas modificaciones con el Diseñador para adaptarlo a las características del formulario de información.
- Y finalmente con las herramientas del framework se crea la presentación.

Crear Meta4Object con Herramienta multinodo

Fase 1 (1 de 4)

Indicar la tabla del nodo raíz que es CEX_BENEFITS.

Fase 1 (2 de 4)

Indicar el identificador del nuevo Meta4Object. En nuestro caso CREX_BE_IN_BENEFITS donde:

- CREX: Cliente Rich
- BE: Módulo de beneficios
- IN: Información
- BENEFITS: funcionalidad del objeto

Revisar que en la sentencia están todos los campos de la tabla con los que se quiere trabajar, en el ejemplo todos.

Fase 1 (3 de 4)

Posicionarse en el nodo raíz.

Como se quiere que el Meta4Object tenga una lista de entrada haga clic en Listas y seleccionamos la lista creada anteriormente llamada CREX_BE_MT_BENEFITS.

Como la tabla de la lista y del nodo raíz es la misma, hay que definir:

- Tipo de relación: 1:1
- Node: CREX_BE_MT_BENEFITS
- Elementos de la lista: CEX_ID_BENEFIT-CEX_ID_BENEFIT

Definir el nodo hijo del nodo raíz: Posicionarse sobre el nodo raíz y hacer clic en **Nuevo**.

Fase 2 (1 de 1)

Llamar al nuevo nodo CREX_BE_IN_H_VALUE_BENEFITS

Seleccionar del listado la tabla CEX_H_VALUE_BENEFITS.

Seleccionar todos los campos.

Se crean automáticamente los conectores entre los campos relacionados de las dos tabla (la del nodo raíz y la del nodo hijo) si están relacionadas por modelo de datos. En el ejemplo no es necesario entrar en la definición de

conectores porque están definidos en el modelo de datos.

Definir el segundo nodo hijo del nodo raíz. Posicionarse sobre el nodo raíz y hacer clic en **Nuevo**.

Fase 2 (1 de 1)

Llamar al nuevo nodo CREX_BE_IN_H_HR_BENEF

Seleccionar del listado la tabla CEX_H_HR_BENEF.

Seleccionar todos los campos menos el comentario porque en la información no es relevante.

Se crean automáticamente los conectores entre los campos relacionados de las dos tabla (la del nodo raíz y la del nodo hijo) si están relacionadas por modelo de datos. En el ejemplo caso no es necesario entrar en la definición de conectores porque están definidos en el modelo de datos.

Fase 1 (4 de 4)

Como no se desea crear la presentación asociada (ya que nos crearía la del multinodo), seleccion Grabar.

Meta4Object con funcionalidad de formulario de información

Tras la creación del Meta4Object CREX_BE_IN_BENEFITS aprovechando la herramienta del Multinodo, es necesario editarlo para hacer una serie de modificaciones:

- indicar que no se ejecute con el asistente (no hay acceso a esta información desde el asistente).
- Posicionarse en el nodo raíz (CREX_BE_IN_BENEFITS) y asignar la propiedad:
 - Nodes Type = 201
- Modificar el tipo de creación del Meta4Object a 21. Esto sólo se debe hacer cuando el Meta4Object se va a utilizar exclusivamente como Meta4Object de información, en caso de reutilización se dejará el 22.
- Hay que crear unos conectores para que se filtren los nodos correctamente
- Nodos hijos:
 - Conectar con el padre la propiedad CUT_OFF_DATE.
- Indicar en el nodo en el que queremos filtrar:
 - Filtro automático: Histórico

- Indicar cual es el nodo cabecera/principal del Meta4Object. Para que se pueda crear un formulario de información para el nuevo Meta4Object hay que indicar cual es el nodo cabecera o el nodo principal. En el ejemplo el nodo cabecera es el nodo principal por lo que sólo es necesario indicar el nodo cabecera, que en nuestro caso es el nodo raíz.

Creación de la presentación

Para la creación de la presentación de formulario de información se utilizan las herramientas del framework:

Herramientas de desarrollo | Desarrollar con el development framework | Crear formulario de información.

Paso1 de 3

La nueva presentación se llama CREX_BE_IN_BENEFITS

Paso2 de 3

Posicionarse en cada nodo y para cada uno de ellos, indicar:

- Si se quiere visualizar en formato formulario o en formato tabla.
- Si se quiere acceder a algún enlace

En el ejemplo se presentarán:

- El nodo CREX_BE_IN_BENEFITS como se ha indicado que es el nodo cabecera no da opción de visualización.
- El nodo CREX_BE_IN_H_VALUES_BENEFITS en formato formulario porque el nodo no tiene más de un registro para un filtro de fechas dado.
- El nodo CREX_BE_IN_H_HR_BENEF en formato tabla porque el nodo puede tener más de un registro para un filtro de fechas dado.

Creación de un filtro

Al Meta4Object de información creado en el ejemplo anterior se le añade un filtro de búsqueda sobre uno de sus nodos. Este filtro se ha creado en el momento de la creación del Meta4Object. El filtro de búsqueda es un Meta4Object de mantenimiento que se va a crear exclusivamente para ser utilizado en este formulario de información.

Se podría utilizar un Meta4Object de mantenimiento ya existente con la única condición de **que la tabla base del nodo a filtrar debe ser la misma que la del Meta4Object de filtro.**

El filtro se aplica sobre el nodo histórico (CREX_BE_IN_H_HR_BENEF) porque este nodo será un nodo que potencialmente podrá tener más de 300 registros (puede haber más de 300 empleados con un beneficio asignado).

El nodo se carga con el máximo número de registros permitido (por defecto 300 aunque parece recomendable rebajar estos nodos a filtrar a 100). Si hay más registros, no se verán todos al entrar, sólo los 300 primeros y se utilizará el filtro para poder ir viendo toda la información.

Crear Meta4Object de filtro

Creamos con las herramientas del development framework un Meta4Object de lista (no hay que generar la presentación) para la tabla base CEX_H_HR_BENEF. Este Meta4Object lo llamaremos CREX_BE_MT_H_HR_BENEF

En el filtro de este Meta4Object se indicarán todos los campos por los que nos va a interesar filtrar el nodo de información.

Modificar el Meta4Object de información

Entramos en el diseñador de Meta4Objects para editar CREX_BE_IN_BENEFITS e indicamos no utilizar el asistente ya que esto es un desarrollo un poco más avanzado. A continuación:

- Se define el filtro
- Se define la carga del mismo:

Definición del filtro

- Crear las propiedades que van a contener las sentencias de filtro:
 - Copiar la definición de los elementos SYS_SENTENCE_A y SYS_PARAM_A del nodo raíz
- Nos posicionamos en el nodo que queremos filtrar (CREX_BE_IN_H_HR_BENEF) y editamos los filtros de nodo. Los valores que indicamos son:
 - Filter ID: CREX_PERIOD_FILTER
 - Other Meta4Object (porque vamos a utilizar un Meta4Object de lista ya existente)
 - Filter type: 3 (search filter, porque no es un filtro de entrada a la presentación sino que es un filtro que se ejecuta bajo demanda)

- Behaviour type ID: 2 (All filters, porque queremos que se muestren todos los filtros del Meta4Object destino, pero no queremos ver la población resultante del filtro hasta que se aplique a la presentación principal, además queremos que se aplique no sólo el filtro de entrada sino la sentencia del Meta4Object que estamos listando).
- Statement item: SYS_SENTENCE_A (al añadir el filtro esta propiedad guardarán la sentencia que se ha generado con el filtro)
- Parameter item: SYS_PARAM_A (al añadir el filtro esta propiedad guardarán los parámetros de carga de la sentencia que se ha generado con el filtro)
- Filter Meta4Object ID: CREX_BE_MT_H_HR_BENEF
- List Node ID: CREX_BE_MT_H_HR_BENEF

Carga con filtro

Se crea un método que será el que se ejecuta después de aplicar filtro

CREX_LOAD_FILTER.

//The properties SYS_SENTENCE_A and SYS_PARAM_A are filled with the filter sentence, then we only need to execute the load.

DestroyBlock()

Load_blk()

Se crea un método que será el que se ejecuta después de borrar el filtro

CREX_REMOVE_FILTER.

//The properties SYS_SENTENCE_A and SYS_PARAM_A are filled with the filter sentence, then we have to empty them and execute the load again

DestroyBlock()

SYS_SENTENCE_A = NullValue()

SYS_PARAM_A = NullValue()

Load_blk()

Presentación

Abrir la presentación del formulario de información CREX_BE_IN_BENEFITS.

Editar las propiedades del objeto de la presentación *O*/Pres2/FormMain/splForm/splForm_Right/panActions/inclcallActionFilterPERIOD_FILTER.

Indicar los métodos anteriores (CREX_LOAD_FILTER y CREX_REMOVE_FILTER) como métodos de añadir filtro y de quitar filtro respectivamente.

Creación de una tabla de valores

Se crea una tabla de valores fijos por empresa: "Valores de la empresa". La creación de tablas de valor de valores incidencias es muy similar mientras que la de incidencias masivas es algo más particular. Para ver los detalles de creación y modificación de estas dos últimas tendrá que consultar el capítulo 4.

La plantilla del framework de tablas de valor consta de un Meta4Object y de una presentación. En los siguientes apartados seguiremos la metodología para:

- Generar el Meta4Object que soporta la carga y edición de valores.
- Generar la presentación que presenta los datos de forma pivotada.



Para ampliar información sobre creación y configuración de tablas de valor puede consultar el anexo A de este manual.

Generar un nuevo Meta4Object a partir de la plantilla

Con el Diseñador de Meta4Objects se crea un nuevo Meta4Object de herencia sobre SRTC_PY_VT_INC_IND (Plantilla de incidencias individuales).

Se guarda el nuevo Meta4Object estableciendo el identificador y el nombre que se desea: SRTC_PY_VT_LEG_ENT y "Valores de la empresa". El nombre sale luego como título de la presentación de la tabla de valor.

Está creada la tabla de valores SCO_LEG_ENT_VALUES (Valores empresa)

SCO LEG ENT VALUES
STD_ID_LEG_ENT
ID_DMD
ID_DMD_COMPONENT
DT_START
DT_COR_START
DT_END
DT_COR_END
<u>SCO_VALUE</u>
SCO_ID_CURRENCY
SCO_DT_EXCHANGE
EX_TYPE
SCO_ID_REASON_CHANGE
<u>ID_PRIORITY</u>
SCO_COMMENT
VT

A continuación se realizan una serie de cambios sobre el Meta4Object SRTC_PY_VT_LEG_ENT creado. Es necesario:

- Definir la población de la tabla de valor
- Configurar la tabla de valor
- Añadir un filtro multiselección sobre la población

Configurar la población

Puesto que las tablas de valores fijos asignan valores a nivel de toda la organización o de periodos del RH o de roles del RH (valores vigentes para más de una paga) es necesario, por tanto, definir una población que representará las filas de nuestra tabla de valores. En nuestro caso elegimos la empresa.

Los pasos a seguir son:

- Sobrescribir el nodo de filas (SRTC_PY_VT_ROWS) con una estructura de nodo nueva con ID SRTC_PT_VT_ROWS_LEG_ENT.
- Editar la sentencia de esta estructura de nodo y:
 - Establecer como tabla base la que corresponda con nuestra población, es decir, con empresas. La tabla es STD_LEG_ENT.

- Añadir los campos de clave primaria y otros posibles que nos puedan interesar mostrar en la fila: STD_ID_LEG_ENT ("ID Empresa"), STD_N_LEG_ENT ("Nombre de empresa"), STD_DT_START ("Inicio") y STD_DT_END ("Fin").
- Ordenar por el campo clave primaria STD_ID_LEG_ENT de forma ascendente.

Configurar la tabla de valores

Sobreescribir el nodo de datos (SRTC_PY_VT_DATA) con una estructura de nodo nueva. Le damos el ID SRTC_PT_VT_DATA_LEG_ENT.

Editar la sentencia de la estructura de nodo y:

- Establecer como tabla base la tabla de valor sobre la que se va a trabajar. Esta tabla estará previamente creada y no su creación no es el objetivo de este ejemplo. En nuestro caso es SCO_LEG_ENT_VALUES ("Valores empresa")
- Añadir todos los campos. Se debe tener en cuenta que existen una serie de reglas para fijar los identificadores de los nombres de los campos en la sentencia. La tabla de valores tenía los identificadores de los campos por defecto y no hay que hacer nada especial. Si no está seguro, por favor, consulte el capítulo 4 Avanzado.
- Editar las propiedades avanzadas del elemento SCO_VALUE ("Valor"), y dentro del apartado 'Propiedades de elementos' establecer a la propiedad Recarga el valor: Rec. elemento para todos los Meta4Objects, con esto conseguiremos un formateo automático de los valores introducidos en la presentación.

Finalmente se guarda la nueva estructura de nodo y se reutiliza en una serie de nodos. Para ello, situarse en cada uno de los nodos, y elegir la opción Sobrescribir nodo, y posteriormente la opción de Seleccionar estructura de nodo existente. Los nodos donde se debe reutilizar son:

- SRTC_PY_VT_CHECK_DATA ("Comprobación de valores")
- SRTC_PY_VT_PIVOT_DATA ("Pivotar datos por fila")
- SRTC_PY_VT_PERSIST ("E.N. de ayuda a la persistencia")

Añadir filtro multiselección sobre la población

Añadir un filtro de nodo multiselección para que al cargar la tabla de valores pregunte sobre cuales empresas vamos a asignar valores.

En el Diseñador de Meta4Objects, editar los filtros de nodo para el nodo de filas SRTC_PY_VT_ROWS (Filas de la tabla de valor). Crear un nuevo filtro y establecer los siguientes valores:

- ID Filtro: FILTRO1 en otro Meta4Object
- Tipo de filtro: 3
- Tipo de comportamiento: 1
- ID Meta4Object de filtro: SRTC_MT_LEG_ENT. El Meta4Object de lista y mantenimiento de empresas SRTC_MT_LEG_ENT se supone ya creado. Si ha elegido otra población (lugar de trabajo, unidad organizativa, etcétera.) tendrá que seleccionar el Meta4Object de lista y mantenimiento correspondiente o crear uno si no lo tiene
- ID Nodo de lista: SRTC_MT_LEG_ENT
- Elemento de la sentencia: SRTC_SYS_SENTENCE
- Elemento de los parámetros: SRTC_SYS_PARAM

Generar una nueva presentación a partir de la plantilla

Desde el Diseñador de presentaciones se crea una nueva presentación que herede de la presentación SRTC_PY_VT_INC_IND que corresponde con la plantilla para valores incidencias.

A continuación se editan las propiedades del objeto raíz de la presentación GeneralValueTable y se cambia la propiedad Idchannel por el identificador del Meta4Object creado anteriormente: SRTC_PY_VT_LEG_ENT.

Para modificar el título del formulario situarse en el explorador de presentaciones en el objeto MainForm (ruta: "**O*/GeneralValueTable/MainForm") y cambiar la propiedad 'Text' para establecer referencia a nuestro Meta4Object:: ##CHNNL[SRTC_PY_VT_LEG_ENT].

Establecer el grupo de dominio

El grupo de dominio define el conjunto de elementos de nómina que serán mostrados inicialmente. Posteriormente pueden ser cambiados desde la ventana de selección de conceptos.

Para establecer el grupo de dominio, situarse en el explorador de presentaciones en el objeto IncludecallsetLevelDMDGroup (ruta "**O*/GeneralValueTable/IncludecallsetLevelDMDGroup") y con el botón derecho establecer las siguientes propiedades:

- Level: 01, que representa el nivel del periodo
- Id_DMD y Id_DMD_GROUP vacíos.

Establecer las claves de la tabla valor en relación con la población

Para establecer las claves situarse en el explorador de presentaciones en el

objeto IncludecallSetPKs (ruta "**O*/GeneralValueTable/IncludecallSetPKs") y con el botón derecho establecer las siguientes propiedades:

- Id_PkA: STD_ID_LEG_ENT

Creación de un asistente de petición de parámetros y ejecución de procesos

Se crea un asistente de petición de parámetros para ejecutar el "Mapa de puestos". Está formado por dos pasos en los que el usuario puede establecer el tipo de salida y una serie de criterios de búsqueda. En el primer paso se pedirá al usuario si se desea visualizar el informe por unidad organizativa o por clase interna de puesto, y en el segundo se pedirán los parámetros descritos a continuación, para permitir al usuario filtrar los datos que quiere mostrar:

- Categoría de puesto: Para filtrar los puestos que pertenecen a esa categoría
- Rango nivel desde y Rango nivel hasta: para definir un rango de niveles del puesto entre los que se encuentren los que van mostrarse en el mapa
- Fecha de vigencia: Para filtrar los datos a una fecha dada
- Unidad organizativa: Para filtrar los puestos de una unidad específica
- Profundidad del árbol: Para establecer un límite de cuantos niveles se quieren filtrar partiendo de la unidad organizativa definida en el parámetro anterior

Los dos últimos sólo se deberán pedir al usuario si se ha seleccionado la opción de visualizar por unidad organizativa.

Requisitos del ejemplo: supone que el proceso a ejecutar (plantilla y proceso de Microsoft Excel) y los Meta4Objects con las reglas de negocio ya existen. El ejemplo consiste en crear los formularios de edición de parámetros que finalmente ejecutarán ese proceso según los criterios establecidos por el usuario.

La plantilla del framework que representa esta funcionalidad se basa en un Meta4Object y en una presentación que el usuario debe heredar y posteriormente configurar para obtener el nuevo componente de tipo asistente.

Por tanto, hay dos pasos principales

- Generar el Meta4Object que soporta las acciones de usuario al navegar por los pasos
- Generar la presentación que representa las pantallas de los dos pasos.

El aspecto final será:

The screenshot shows a dialog box titled "Selección tipo de salida" (Output type selection) with a progress indicator "[1/2]". It contains two radio button options: "Por Clase interna de puesto" (By internal job class) and "Por Unidad organizativa" (By organizational unit). The "Por Unidad organizativa" option is selected. At the bottom, there are four buttons: "Cancelar" (Cancel), "< Anterior" (Previous), "Siguiente >" (Next), and "Ejecutar" (Execute).

The screenshot shows a dialog box titled "Criterios de búsqueda" (Search criteria) with a progress indicator "[2/2]". It contains several input fields: "ID Work Unit" (a text box followed by a dropdown arrow), "ID Job Category" (a text box followed by a dropdown arrow), "Tree depth" (a text box), "Date" (a text box followed by a calendar icon), "Range from" (a text box with the value "1"), and "Range to" (a text box with the value "5"). At the bottom, there are four buttons: "Cancelar" (Cancel), "< Anterior" (Previous), "Siguiente >" (Next), and "Ejecutar" (Execute).

A continuación se detallan los pasos necesarios en la construcción del

asistente de petición de parámetros.



Para ampliar información sobre creación de asistentes de petición de parámetros puede consultar el anexo B de este manual.

Generación del Meta4Object

Desde el Diseñador de Meta4Objects se crea un nuevo Meta4Object de herencia sobre SRTC_PARAM_TEMPLATE ("Parameter request template") con:

- Meta4Object ID = SRCO_OR_RP_JOB_LEVEL
- Meta4Object Name = Job Level

Crear los nodos para los pasos del asistente

Crear un nuevo nodo libre por cada paso.

- Para el primer paso, establecemos
 - ID Node = SRCO_OR_RP_JOB_LEVEL_PANEL_1
 - Node Name = Job Level Step 1
 - ID Node structure = SRCO_OR_RP_JOB_LEVEL_PANEL_1
- Para el segundo paso:
 - ID Node = SRCO_OR_RP_JOB_LEVEL_PANEL_2
 - Node Name = Job Level Step 2
 - ID Node structure = SRCO_OR_RP_JOB_LEVEL_PANEL_2

Añadir los parámetros de cada paso

Por cada paso, los parámetros se presentan como elementos de ámbito bloque en cada uno de los nodos.

Así, para el paso 1 basta con una propiedad tipo numérica (1, 0) para almacenar el tipo de vista (Por clase interna o por unidad organizativa). Crearemos la propiedad SRCO_VIEW_TYPE ("Job Level Type"). Si el valor es 1 será "Por clase interna" y si es 0 será "Por unidad organizativa". Hay que tener en cuenta que en función de este valor, el asistente tendrá que más o menos parámetros en el siguiente paso. Eso se trata más adelante cuando se crea la presentación.

En el **paso 2** se crean las propiedades siguientes:

- Para facilitar que el usuario introduzca la "Categoría puesto" conviene mostrar un grupo lista validación de forma que tenga un acceso a la lista y que si introduce un ID la presentación se lo valide. Por tanto, para ello se crean dos propiedades una para almacenar el ID y otra para el nombre:
 - SRCO_ID_JOB_CATEGORY ("ID Job Category") de tipo Cadena (30)
 - SRCO_N_JOB_CATEGORY ("Job Category Name") de tipo Cadena (50) (*)
- SRCO_RANGE_FROM ("Rango nivel desde") de tipo Numérico (18)
- SRCO_RANGE_TO ("Rango nivel hasta") de tipo Numérico (18).
- SRCO_DATE ("Fecha de vigencia") de tipo fecha
- Para facilitar que el usuario introduzca la "Unidad organizativa" conviene mostrar un grupo lista validación de forma que tenga un acceso a la lista y que si introduce un ID la presentación se lo valide. Por tanto, para ello se crean dos propiedades una para almacenar el ID y otra para el nombre:
 - SRCO_ID_WORK_UNIT ("ID Work Unit") de tipo Cadena (30)
 - SRCO_N_WORK_UNIT ("Work Unit name") de tipo Cadena (50) (*)

(*) esas dos propiedades serán necesarias únicamente para que el asistente puede mantener los nombres al navegar entre los pasos, por si hay que volver a un paso anterior o posterior y se tenía ya una selección hecha.

Generar los métodos de carga de cada paso

Un paso requiere un método de carga de datos específico cuando al acceder al mismo se quieren establecer valores que dependen de los datos seleccionados en otros pasos.

Para establecer valores iniciales "fijos" a los parámetros no es necesario generar un método de carga. Se recomienda utilizar los valores por defecto de las propiedades creadas en el paso anterior. Por tanto, en nuestro ejemplo no será necesario generar un método de carga para ningún paso.

Establecer ciertos valores por defecto:

- En el primera paso, se marca la opción de visualizar Por clase interna de puesto, es decir estableceremos la propiedad Valor por defecto de la propiedad SRCO_VIEW_TYPE con el valor 1.
- En el segundo, los valores 1 y 5 para Rango desde y Rango hasta y 2 para Depth Tree.

Generar los métodos de validación de cada paso

Un paso requiere un método de validación de datos específico cuando se

desea realizar validaciones funcionales y de tipo obligatoriedad sobre los valores de los parámetros cuando el usuario sale del paso, tras hacer clic en Anterior, Siguiente o Finalizar.

Para ilustrar el ejemplo, se crea un método de validación en el segundo paso para comprobar que el usuario introduce un valor en "Rango desde" y "Rango hasta" comprendido entre 1 y 10.

Para ello se genera un método de ámbito nodo con el identificador: VALIDATE_STEP_PANEL_2 sobre el nodo LOC_PARAM_TEMPLATE del Meta4Object. Por tanto, se sobrescribe el nodo y se genera una nueva estructura de nodo con el ID SRCO_OR_RP_JOB_LEVEL_LOC_PARAM.

PANEL_2 debe coincidir con el alias definido para el panel del segundo paso asociado al paso en la presentación OBL que después se genere. Para esto se copia del método VALIDATE_STEP_ que ya aparece en el nodo.

El código del método VALIDATE_STEP_PANEL_2 podría ser :

```
bValidate = M4_TRUE
If IsNull (SRCO_OR_RP_JOB_LEVEL_PANEL_2.SRCO_RANGE_FROM) = M4_FALSE Then
    If SRCO_OR_RP_JOB_LEVEL_PANEL_2.SRCO_RANGE_FROM > 10 Or
    SRCO_OR_RP_JOB_LEVEL_PANEL_2.SRCO_RANGE_FROM < 1 Then
        bValidate = M4_FALSE
    EndIf
EndIf

if bValidate = M4_FALSE Then
    MessageBox ("Job level", " Range from must be set from 1 to 10")
    return M4_FALSE
EndIf

bValidate = M4_TRUE
If IsNull (SRCO_OR_RP_JOB_LEVEL_PANEL_2.SRCO_RANGE_TO) = M4_FALSE Then
    If SRCO_OR_RP_JOB_LEVEL_PANEL_2.SRCO_RANGE_TO > 10 And
    SRCO_OR_RP_JOB_LEVEL_PANEL_2.SRCO_RANGE_TO < 1 Then
        bValidate = M4_FALSE
    EndIf
EndIf

if bValidate = M4_FALSE Then
    MessageBox ("Job level", " Range to must be set from 1 to 10")
    return M4_FALSE
EndIf
```

```
return M4_TRUE
```

Establecer el método de ejecución y cancelación

Dependiendo del tipo de proceso a ejecutar es posible que necesite ejecutar algún método como parte de su proceso cuando el usuario haga clic en el botón Ejecutar.

En el ejemplo, el proceso a ejecutar es un método del Meta4Object cuyo ID es SRCO_OR_RP_GENERATE_EXCEL ("Categoría de puestos"). El método SRCO_GENERATE_EXCEL del nodo SRCO_OR_RP_GENERATE_EXCEL genera un archivo de tipo Microsoft Excel basado en una plantilla. No es objetivo de este ejemplo definir este proceso. Tan solo basta tener en cuenta que para que el método SRCO_GENERATE_EXCEL funcione es necesario establecer valores sobre una serie de propiedades del nodo SRCO_OR_RP_GENERATE_EXCEL. En concreto, se deberán establecer valores sobre:

- SRCO_DATE
- SRCO_ID_JOB_CATEGORY
- SRCO_RANGE_FROM
- SRCO_RANGE_TO
- SRCO_VIEW_TYPE
- SRCO_ID_WORK_UNIT
- SRCO_DEPTH_TREE

En este caso, para poder invocar este método que generará el archivo Excel se sobrescribe el nodo LOC_PARAM_TEMPLATE y se genera una nueva estructura de nodo si no se generó antes. Para ello, se crea la ID Estructura de nodo SRCO_OR_RP_JOB_LEVEL_LOC_PARAM.

A continuación sobrecribir la regla del método EXECUTE_PROCESS y establecer lo que se quiera realizar cuando se pulsa el botón de Ejecutar desde el formulario de petición de parámetros. En este caso para poder invocar al método que genera el archivo Excel del Meta4Object SRCO_OR_RP_GENERATE_EXCEL será necesario crear un alias de Meta4Object previamente. Llamar al alias "VIEW_EXCEL". El código de la regla sobrescrita puede ser algo del estilo:

```
If IsNull(SRCO_OR_RP_JOB_LEVEL_PANEL_2.SRCO_DATE) = True Then
    VIEW_EXCEL!SRCO_OR_RP_GENERATE_EXCEL.SRCO_DATE = DxDate(01,01,4000)
Else
```

```

VIEW_EXCEL!SRCO_OR_RP_GENERATE_EXCEL.SRCO_DATE =
SRCO_OR_RP_JOB_LEVEL_PANEL_2.SRCO_DATE
Endif

If SRCO_OR_RP_JOB_LEVEL_PANEL_1.SRCO_VIEW_TYPE = "1" Then
    SRCO_OR_RP_JOB_LEVEL_PANEL_2.SRCO_ID_WORK_UNIT = NullValue()
    SRCO_OR_RP_JOB_LEVEL_PANEL_2.SRCO_DEPTH_TREE = NullValue()
Endif

VIEW_EXCEL!SRCO_OR_RP_GENERATE_EXCEL.SRCO_ID_JOB_CATEGORY =
SRCO_OR_RP_JOB_LEVEL_PANEL_2.SRCO_ID_JOB_CATEGORY
VIEW_EXCEL!SRCO_OR_RP_GENERATE_EXCEL.SRCO_ID_WORK_UNIT =
SRCO_OR_RP_JOB_LEVEL_PANEL_2.SRCO_ID_WORK_UNIT
VIEW_EXCEL!SRCO_OR_RP_GENERATE_EXCEL.SRCO_RANGE_FROM =
SRCO_OR_RP_JOB_LEVEL_PANEL_2.SRCO_RANGE_FROM
VIEW_EXCEL!SRCO_OR_RP_GENERATE_EXCEL.SRCO_RANGE_TO =
SRCO_OR_RP_JOB_LEVEL_PANEL_2.SRCO_RANGE_TO
VIEW_EXCEL!SRCO_OR_RP_GENERATE_EXCEL.SRCO_VIEW_TYPE =
SRCO_OR_RP_JOB_LEVEL_PANEL_1.SRCO_VIEW_TYPE
VIEW_EXCEL!SRCO_OR_RP_GENERATE_EXCEL.SRCO_DEPTH_TREE =
SRCO_OR_RP_JOB_LEVEL_PANEL_2.SRCO_DEPTH_TREE

If VIEW_EXCEL!SRCO_OR_RP_GENERATE_EXCEL.SRCO_GENERATE_EXCEL() = M4_ERROR Then
    Return M4_FALSE
Endif

Return M4_TRUE

```

Donde se puede ver como se pasan los valores de los parámetros y finalmente se ejecuta el método SRCO_GENERATE_EXCEL. El método debe devolver M4_TRUE si la ejecución ha sido correcta y M4_FALSE en caso contrario.

Generar la presentación y establecer parámetros globales

Desde el Diseñador de presentaciones se crea una nueva presentación de tipo Formulario que hereda sobre el ID Presentación SRTC_PARAM_TEMPLATE_PT ("Parameter request template") y se guarda con el ID Presentación SRCO_OR_RP_JOB_LEVEL_PT.

A continuación seleccionar en el árbol Explorador de presentaciones el objeto "Presentation1" y hacer clic en el botón derecho del ratón para elegir la opción "Propiedades". Cambiamos el valor de la propiedad "Idchannel" para indicar el ID Meta4Object que ha generado en el apartado anterior: SRCO_OR_RP_JOB_LEVEL.

Finalmente establecer los parámetros globales de la misma. Para ello nos

situamos en el árbol Explorador de proyectos sobre el objeto "inclInitPresentation" y editar sus propiedades:

- En ARG_STEP_ORDER_LIST establecer los pasos del asistente. Los nombres que ahí se establezcan se identifican posteriormente con alias de objetos tipo panel que se crean más adelante. La separación entre pasos debe ser el carácter almohadilla (#). Se recomienda que conserve la notación que se le sugiere: para nuestros dos pasos PANEL_1#PANEL_2. Posteriormente se definen los dos paneles.
- En ARG_STEP_TO_ENABLE_EXE_BUTTON indicar el paso a partir del cual se debe habilitar el botón de ejecución: PANEL_2
- En ARG_ASK_DYN_FILTER indicar si quiere solicitar el filtro dinámico (1: Si, 0: No). Para nuestro ejemplo pondremos 0.
- El resto de parámetros toman el valor por defecto. Para obtener más información sobre los mismos consulte al [capítulo avanzado].

A continuación, repetir la operación sobre el objeto "Form1": situarse sobre él y elegir la opción "Propiedades". En la propiedad "Text" sustituir lo que está entre corchetes por el ID Meta4Object que ha generado en el apartado anterior: ##CHNNL[SRCO_OR_RP_JOB_LEVEL].

Generar los paneles asociados a cada paso

Por defecto la presentación creada tiene dos paneles vacíos generados que servirán para el ejemplo. En general se deberán tener tantos paneles distintos como pasos necesite la petición de parámetros. Todos los paneles deben aparecer dentro del componente ChangerSteps que se encuentra bajo Form1.

Por cada paso rellenar la propiedad IdNode del panel con identificador del nodo asociado al paso en el Meta4Object de petición de parámetros generado. En nuestro caso para el panel PANEL_1 establecer SRCO_OR_RP_JOB_LEVEL_PANEL_1 y para el panel PANEL_2 establecer SRCO_OR_RP_JOB_LEVEL_PANEL_2.

A continuación configurar el título y descripción de cada panel. Esto se realiza asignando el título y descripción sobre los argumentos ARG_TITLE y ARG_DESCRIPTION del include SRTC_INCL_PARAM_TMPLT_HEADER que tiene como alias inclParamHeader y que está situado dentro de cada panel. También hay que tener en cuenta si el título o descripción debe ser válido en varios idiomas. Si es así, se recomienda que utilice el nombre del nodo asociado al paso para almacenar el título y que cree un elemento en los nodos de cada paso para la descripción y establecer en el nombre del elemento la descripción (limitación de 50 caracteres). Si se necesitan descripciones mayores de 50 caracteres y multiidioma hay que crear un mensaje en el sistema de mensajes de error y recuperarlo con la función LN4 GetLog en un concepto. En el ejemplo no se indica descripción por ser cada uno de los pasos muy claros pero puede ver como hacerlo en el capítulo avanzado.

Así, en el paso 1 (PANEL_1) para pasar el nombre del nodo y el nombre del elemento como valores de los argumentos, editar las propiedades del include inclParamHeader:

- ARG_TITLE = ##ND[SRCO_OR_RP_JOB_LEVEL_PANEL_1] Hay que modificar el nombre del nodo SRCO_OR_RP_JOB_LEVEL_PANEL_1 para que indique el nombre del paso "Selección del tipo de salida"
- ARG_DESCRIPTION = ##ND[NODO.ITEM]

Para el paso 2:

- ARG_TITLE = ##ND[SRCO_OR_RP_JOB_LEVEL_PANEL_2] .Hay que modificar el nombre del nodo SRCO_OR_RP_JOB_LEVEL_PANEL_2 para que indique el nombre del paso *Criterios de búsqueda*
- ARG_DESCRIPTION=##TM[nombre_del_item_que_contenga_la_descripción_en_su_nombre_o_concepto_que_lo_calcule]

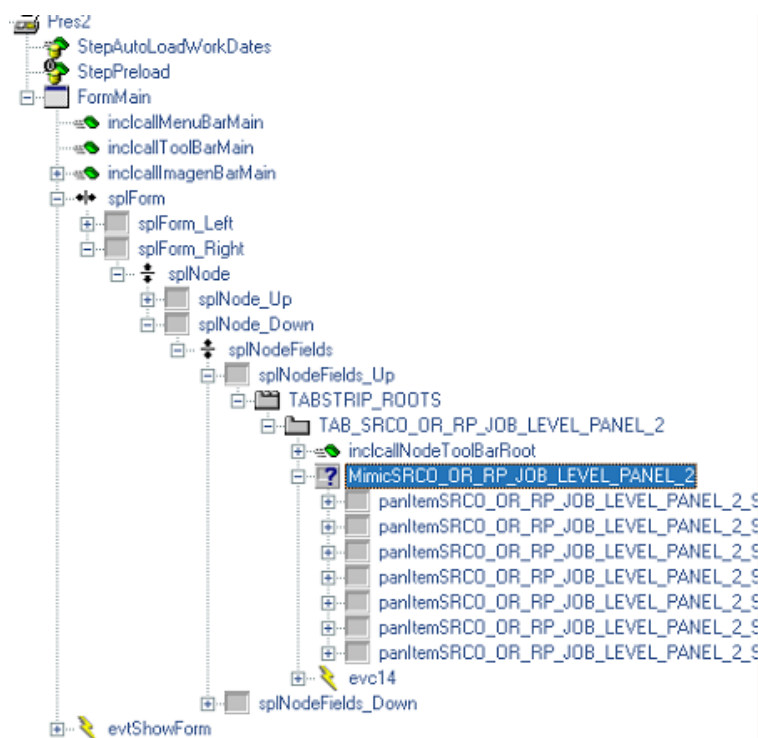
Modificar los paneles de cada paso

A continuación hay que añadir los objetos que necesite en cada panel para realizar la petición de parámetros deseados. Estos objetos irán sobre los elementos propiedad de ámbito bloque generados en el nodo asociado al paso en el Meta4Object de petición de parámetros.

Una forma rápida de hacer esto, dado que normalmente coinciden con todos los elementos propiedad generados en el nodo, es poner los elementos en el nodo como visibles y generar una presentación por defecto de tipo "Mantenimiento" sobre dicho nodo y luego copiar los controles generados sobre nuestra presentación.

Para el paso 1, se utiliza un objeto Checkboxgroup para crear dos opciones para poder seleccionar "Por clase interna de puesto" o "Por unidad organizativa". Si el usuario selecciona la primera se deberá almacenar en la propiedad SRCO_VIEW_TYPE del nodo SRCO_OR_RP_JOB_LEVEL_PANEL_1 el valor "1" y si elige "Por unidad organizativa" se almacenará "0".

Para el paso 2 se ponen las propiedades del nodo SRCO_OR_RP_JOB_LEVEL_PANEL_2 como visibles y se generan una presentación por defecto de tipo "Mantenimiento". En la imagen de abajo, se ve la estructura de la presentación "Mantenimiento" generada y desde donde se copian los controles: todos los contenidos en el MimicSRCO_OR_RP_JOB_LEVEL_PANEL_2. Para realizarlo, lo más cómodo es posicionarse en el explorador de presentaciones sobre el Mimic y editar el código OBL. De ahí se copia el código y se paga bajo el PANEL_2 de la presentación.



Después de ese primer paso, hay que modificar cada panel a mano para añadir los grupos listas validación o los controles particulares necesarios.

Para el paso 2 se quiere tener un aspecto del estilo

En el ejemplo, el grupo lista validación asociado al campo ID Job Category requiere de la existencia del Meta4Object de mantenimiento SRCO_OR_MT_JOB_CATEGOR. Así el código OBL del panel de edición al ID Job Category sería:

```
BEGIN Panel panItemSRCO_OR_RP_JOB_PANEL_1_SRCO_ID_WORK_UNIT
  Idchannel = "SRCO_OR_RP_JOB_LEVEL"
  Idnode = "SRCO_OR_RP_JOB_LEVEL_PANEL_2"
```

```

Top = 66
Width = 710
Height = 28
Align = "Top"
BorderStyle = 0
Visible = False
BEGIN Label zooSRCO_ID_WORK_UNIT
    Left = 8
    Top = 6
    Width = 162
    Height = 20
    Text = "##TM[SRCO_ID_WORK_UNIT]"
    Alignment = 1
END
BEGIN Textbox txtSRCO_ID_WORK_UNIT
    Iditem = "SRCO_ID_WORK_UNIT"
    Left = 174
    Top = 4
    Width = 90
    Height = 20
    Dependson = "SRCO_ID_WORK_UNIT"
    Collabel = "##TM[SRCO_ID_WORK_UNIT]"
    Colfontbold = True
    Tabindex = 0
    Format = "Private"
BEGIN Evvalidate evvalSRCO_ID_WORK_UNIT14
    BEGIN Action_call actSRCO_ID_WORK_UNIT15
        Sentence = "**O*../../stgrpSRCO_ID_WORK_UNIT.Validate"
    END
END
BEGIN Evdblclick evcdlbSRCO_ID_WORK_UNIT16
    BEGIN Action_call actSRCO_ID_WORK_UNIT17
        Sentence = "**O*../../stgrpSRCO_ID_WORK_UNIT.List"
    END
END
END
BEGIN Textbox txtSTD_N_WORK_UNIT
    Iditem = "SRCO_N_WORK_UNIT"
    Left = 268
    Top = 4
    Width = 275

```

```

Height = 20
BackColor = -2147483633
ForeColor = 0
Dependson = "SRCO_ID_WORK_UNIT"
Colfontbold = True
Editable = False
Format = "Private"
Vbcompulsory = True
BEGIN Evdblclick evcdlbSTD_N_WORK_UNIT18
  BEGIN Action_call actSTD_N_WORK_UNIT19
    Sentence = "**O*../../lstgrpSRCO_ID_WORK_UNIT.Validate([*F*False])"
  END
END
END
BEGIN Button lstSRCO_ID_WORK_UNIT
  Iditem = "SRCO_ID_WORK_UNIT"
  Left = 545
  Top = 4
  Width = 20
  Height = 20
  Image = "#List"
  BEGIN Evclick evcSRCO_ID_WORK_UNIT20
    BEGIN Action_call actSRCO_ID_WORK_UNIT21
      Sentence = "**O*../../lstgrpSRCO_ID_WORK_UNIT.List"
    END
  END
END
END
BEGIN Listgroup lstgrpSRCO_ID_WORK_UNIT
  Iditem = "SRCO_ID_WORK_UNIT"
  Watchobject = "**O*../txtSRCO_ID_WORK_UNIT.Visible"
  BEGIN Function List
    Idfuncchannel = "SRCO_OR_MT_WORK_UNIT"
    Idfuncnode = "SRCO_OR_MT_WORK_UNIT"
    Idfuncmethod = "LIST"
    Appiniarg = "SCO_DT_START"
    Appfinarg = "SCO_DT_END"
    Mainargs = "ARG_STD_ID_WORK_UNIT"
    Mainobjs = "SRCO_ID_WORK_UNIT"
    Targetitems = "STD_ID_WORK_UNIT,STD_N_WORK_UNIT"
    Targetobjs = "SRCO_ID_WORK_UNIT,*O*../txtSTD_N_WORK_UNIT.Text"
  BEGIN Presentation Preload

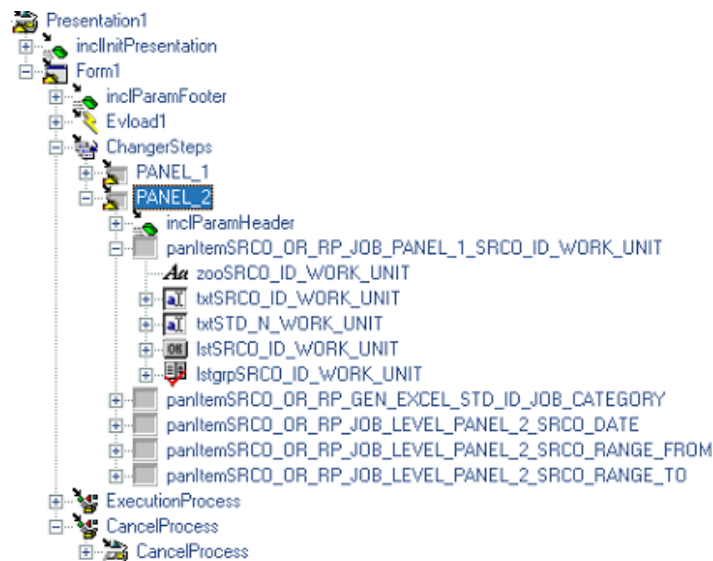
```

```

BEGIN Action_preload extDefault22
    Buttonnew = False
    Availablesteps = 12
    Loadchannel = False
    Onok = "END_OK"
    Oncancel = "StepUnloadForm"
END
END
BEGIN Presentation MetaObject
    BEGIN Extends extDefault23
        Idchannel = "SRCO_OR_MT_WORK_UNIT"
        Idnode = "SRCO_OR_MT_WORK_UNIT"
        Iditem = "SRCO_ID_WORK_UNIT"
        Idinstance = "#Default_list"
        Inheritdata = 1
    END
END
END
BEGIN Function Validate
    Idfuncchannel = "SRCO_OR_MT_WORK_UNIT"
    Idfuncnode = "SRCO_OR_MT_WORK_UNIT"
    Idfuncmethod = "VALIDATE"
    Appiniarg = "SCO_DT_START"
    Appfinarg = "SCO_DT_END"
    Sourcenullargs = "List"
    Mainargs = "ARG_STD_ID_WORK_UNIT"
    Mainobjs = "SRCO_ID_WORK_UNIT"
    Targetitems = "STD_N_WORK_UNIT"
    Targetobjs = "**O*.././txtSTD_N_WORK_UNIT.Text"
END
BEGIN Function Clear
    Targetobjs = "**O*.././txtSTD_N_WORK_UNIT.Text"
END
END
END
asdsd

```

La estructura de la presentación quedaría similar a



Establecer el proceso a ejecutar

Finalmente hay que configurar en la presentación el proceso a ejecutar. En el ejemplo se trataba de la ejecución de un método de un Meta4Object. Como la opción por defecto que presenta la plantilla es la ejecución de un informe hay que realizar la siguiente operación sobre el componente de tipo ExeBlocks ExecutionProcess: sustituir el contenido bajo ExecutionProcess por el contenido de la presentación del proceso a ejecutar:

```
BEGIN Presentation ExecutionProcess

BEGIN Includecall inclExecutionMethod
  Idinclude = "SRTC_INCL_EXECUTE_METHOD"
END

BEGIN Action_script CheckExecution
  Code = "if ExecutionResult= 1 then return 1 else return 0"
  Ontrue = "END_PRESENTATION"
  Onfalse = "END_PRESENTATION"
  Sourcenames = "ExecutionResult"
  Sourceobjects = "*C*EXECUTION_RESULT"
END
END
```

Para realizarlo, la recomendación es situarse en el explorador de presentaciones sobre el ExeBlocks ExecutionProcess y editar el código OBL y realizar los cambios a mano.

Finalmente se define el comportamiento de la petición de parámetros en función de si la ejecución de proceso ha sido correcta o errónea. (Bloque Action_script CheckExecution). Hay que indicar bajo los eventos Ontrue lo que hay que hacer si todo va bien y bajo el evento Onfalse lo que hay que hacer en caso de cancelación o ejecución errónea del proceso. Importante: sólo se deben modificar las propiedades Ontrue y Onfalse.

En el ejemplo si todo va bien se quiere que se cierre la presentación (actUnload) y sino se mantenga la presentación abierta. Para ello ponemos el siguiente código en la Propiedad Ontrue del Action_script CheckExecution:

```
BEGIN Presentation ExecutionProcess

  BEGIN Includecall inclExecutionMethod
    Idinclude = "SRTC_INCL_EXECUTE_METHOD"
  END

  BEGIN Action_script CheckExecution
    Code = "if ExecutionResult= 1 then return 1 else return 0"
    Ontrue = "actUnload"
    Onfalse = "END_PRESENTATION"
    Sourcenames = "ExecutionResult"
    Sourceobjects = "**C*EXECUTION_RESULT"
  END

  BEGIN Action_call actUnload
    Sentence = " *O*/Presentation1/Form1.Unload(END_OK)"
  END
END
```

Creación de un calendario en una presentación

La creación de un calendario no responde a los mismos patrones del resto de plantillas. En este caso para poder utilizarlo es necesario integrarlo en una presentación. La integración del calendario en una presentación es a través de una componente de tipo urlbox, a la que se establece en la propiedad url la llamada a la página HTML del calendario junto con los parámetros correspondientes.

La url debe tener la siguiente forma:

Ruta/lenguaje_ejecución/calendar.htm?param1=val1&... paramn= valn

Donde:

- **Ruta** es el directorio donde se encuentra ubicada la página calendar.htm. La forma de obtener esta ruta es a través de la variable de contexto *V*MENU_HTML_PATH. Esta variable de contexto devuelve la página precedida por la directiva http. Si se está trabajando en el cliente de desarrollo hay que anteponer la directiva file:// a la ruta para evitar problemas de longitud de la url.

Ejemplo de obtención de la ruta sobre la variable de contexto sPathPage:

```
BEGIN Action_script asPathPage
Code = "If IndexOf(ConvertCase(ARG_PATH,M4_UPPERCASE), "HTTP", 0) = 0
Then Return(ARG_PATH) Else Return("file://" + ARG_PATH)"
Returnobj = "C*sPathPage"
Sourcenames = "ARG_PATH"
Sourceobjects = "**V*MENU_HTML_PATH"
END
```

- **lenguaje_ejecución** es el identificador numérico correspondiente al idioma de ejecución. La forma de obtener este idioma es pidiéndolo al Meta4Object SCH_SESSION. Esto se puede realizar a nivel de Meta4Object, obteniendo en un elemento el valor, o añadiendo una componente de tipo textbox en el formulario que lea este parámetro del Meta4Object SCH_SESSION como sigue:

Ejemplo de obtención del lenguaje de ejecución sobre el textbox txtLanguage

```
BEGIN Textbox txtLanguage
Idchannel = "SCH_SESSION"
Idnode = "ROOT_SESSION"
Iditem = "LANGUAGE"
Idchannelkey = "SESSION_CHANNEL"
Left = 466
Top = 92
Width = 146
Height = 20
Visible = False
Text = "Language - do not remove!"
END
```

Para asignar la url al urlbox se supone que existe una componente de tipo urlbox en un formulario, por ejemplo:

```
BEGIN Urlbox urlboxCalendar
Top = 68
```

```

Width = 810
Height = 539
Align = "Bottomfixed"
END

```

La asignación de la url sobre la componente urlbox que mostrará el calendario se puede hacer por ejemplo en el evento load del formulario. A continuación se ve un ejemplo de esta asignación. En este ejemplo, la url con los parámetros se está obteniendo de un elemento SRCO_GET_URL_STRING cuyo valor será del estilo de:

calendar.htm?StartMonth=10&StartYear=2005&NumberOfMonths=3.

```

BEGIN Evload Evload1
    BEGIN Action_call acRefreshCalendar
        Sentence = "**O*/Presentation1/FormMain/Exeblocks.Call(prsRefreshCalendar)"
    END
END

BEGIN Presentation prsRefreshCalendar
    BEGIN Action_script asPathPage
        Code = "If IndexOf(ConvertCase(ARG_PATH,M4_UPPERCASE), "HTTP", 0) = 0
            Then Return(ARG_PATH) Else Return("file://" + ARG_PATH)"
        Returnobj = "**C*sPathPage"
        Sourcenames = "ARG_PATH"
        Sourceobjects = "**V*MENU_HTML_PATH"
    END

    BEGIN Action_method amGetUrlString
        Iditem = "SRCO_GET_URL_STRING"
        Returnobj = "**V*SRCO_PRP_CALENDAR_PARAMS"
    END

    BEGIN Action_set asURLString
        Set = "**V*SRCO_PRP_URL_STRING:=["*C*sPathPage]["*O*../../
txtLanguage.Text]["*V*SRCO_PRP_CALENDAR_PARAMS]"
    END

    BEGIN Action_set asUrlCalendar
        Set = "**O*../../urlboxCalendar.Url:=["*V*SRCO_PRP_URL_STRING]"
    END
END

```

Creación de un desktop

Se crea desktop para el módulo de evaluación: *Desktop de Evaluación*. Este desktop se va a generar sobre la plantilla global.

La plantilla del framework de desktops consta de un Meta4Object y de una presentación. En los siguientes apartados seguiremos la metodología para:

- Generar el Meta4Object que soporta el desktop.
- Generar la presentación que presenta los datos del desktop.
- Configurar el desktop
- Generar tarea de acceso al desktop



Para ampliar información sobre creación y configuración de desktops puede consultar el anexo C de este manual.

Generar un nuevo Meta4Object a partir de la plantilla

Con el Diseñador de Meta4Objects se crea un nuevo Meta4Object de herencia sobre SRCO_GN_DESKTOP (Plantilla global de desktops).

Se guarda el nuevo Meta4Object estableciendo el identificador y el nombre que se desea: SRCO_EV_DESKTOP y *Desktop de evaluación*.

Generar una nueva presentación a partir de la plantilla

Desde el Diseñador de presentaciones se crea una nueva presentación que herede de la presentación SRCO_GN_DESKTOP que corresponde con la plantilla global de desktops.

A continuación se editan las propiedades del objeto raíz de la presentación *PresMain* y se cambia la propiedad *Idchannel* por el identificador del Meta4Object creado anteriormente: SRCO_EV_DESKTOP.

Se guarda la nueva presentación estableciendo el identificador y nombre que se desea: SRCO_EV_DESKTOP.

Configurar el desktop

Para configurar un desktop seleccione la opción:

Herramientas de desarrollo | Desarrollar con el development framework | Configuración de desktops

A continuación, proceda a generar una nueva configuración indicando el identificador y el nombre que se desea: EVALUATION_DESKTOP (Desktop de Evaluación). El nombre indicado será el que posteriormente aparezca al ejecutar el desktop.

Por último asocie a dicho desktop un conjunto de opciones de menú, procesos de negocio, informes, tareas de informes y gráficos relacionadas con el módulo de evaluación y se proceda a grabar la configuración.

Generar tarea de acceso al desktop

Desde el Diseñador de tareas se crea una nueva tarea con el wizard genérico, se establece el identificador y nombre que se desea:

BP_EVALUATION_DESKTOP (Tarea del desktop de evaluación).

A continuación se procede a pasar los siguientes pasos del asistente hasta llegar al paso de edición de código (Paso 4 de 4). Se selecciona la opción de código Windows y se procede a editar el código. En el código se indica la presentación a ejecutar, que se corresponderá con la del desktop (SRCO_EV_DESKTOP), y el nombre de la configuración a mostrar (EVALUATION_DESKTOP). El código será de la forma:

```
StForm.PTCMFTask
```

```
SetConstStr #SYS_IDINSTANCE#, "SRCO_EV_DESKTOP";
```

```
SetConstStr #ID_DESKTOP_CONF#, "EVALUATION_DESKTOP";
```

Por último se pulsa el botón de finalizar para grabar e indicar la tarea como correcta.

A partir de este momento la ejecución de esta tarea mostrará el desktop que se acaba de crear, esta tarea se puede por ejemplo lanzar desde una opción de menú.

Configuración avanzada

3

Acerca de este capítulo

En este capítulo se describen una serie de conceptos avanzados del framework cuyo conocimiento no es imprescindible para el desarrollo de objetos básicos. Sin embargo su conocimiento es de gran utilidad para casos concretos.

Se recomienda revisar las introducciones y apartados generales para tener presente en que momento del desarrollo pueden ser de utilidad.

En concreto, en este apartado se verá:

- Estructura y funcionamiento de las plantillas
- Metodología de desarrollo avanzada
- Transferencia de datos entre Meta4Objects



Para saber más sobre la *configuración de tabla de valores*, *asistentes de petición de parámetros* y *ejecución de procesos y desktops* puede consultar los anexos **A**, **B** y **C** de este manual.

Estructura y funcionamiento de las plantillas

Las plantillas del framework están formadas principalmente por Meta4Objects y presentaciones y la metodología propone usar la herencia para crear nuevos objetos o modificar existentes.

El tipo de herencia que implanta el framework está basado en los modelos de herencia de Meta4Objects de estructuras de nodo, de tablas y de sentencias de la tecnología Meta4.

Se considera que ésta es mono herencia, es decir, cada objeto que se hereda o

se deriva sólo lo puede hacer de un único objeto padre. Sin embargo, es posible derivar finalmente de más de un objeto si el padre deriva de otro objeto.

El modelo de herencia está basado en la herencia por inclusión, es decir, un hijo que deriva de otro adquiere todos los miembros de su padre, sin poder excluir ninguno de ellos. La herencia por inclusión tiene una serie de limitaciones que obligan al desarrollador a plantear el diseño de manera que no sea necesaria hacer exclusión de componentes. El árbol de herencia deberá desarrollarse en función de esta característica.

El development framework aprovecha la funcionalidad de la herencia ofreciendo una serie de ventajas al desarrollador:

- Reutilización de componentes: se pueden desarrollar y modelizar distintos componentes para reutilizarlos en distintas partes.
- Mantenimiento: cuando se produce un cambio en la clase padre, se propaga automáticamente a la clase hija.
- Modularización: el código es mucho más sencillo, ya que quedan separadas las características que son genéricas de las que son específicas.

Los Meta4Objects de las plantillas de lista y mantenimiento, multinodo y formularios de información siguen un esquema de herencia tanto a nivel de Meta4Object como de estructura de nodo que describimos a continuación.

Árbol de herencia de Meta4Objects

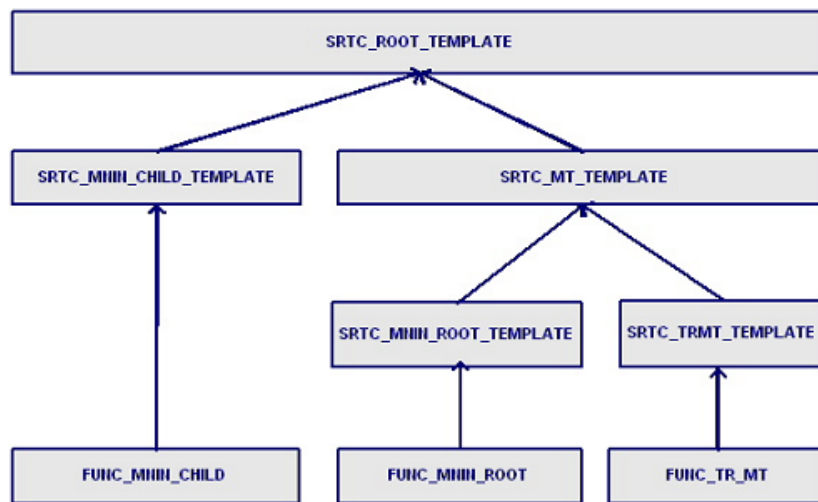
El árbol de herencia tiene las siguientes características:

- Los Meta4Objects de lista y mantenimiento heredan del Meta4Object SRTC_MT_ROOT.
- Los Meta4Objects multinodos y de formularios de información heredan del Meta4Object SRTC_MNIN_ROOT.

Árbol de herencia de estructuras de nodo de datos

Para las estructuras de nodo de datos, el árbol de herencia se representa de izquierda a derecha donde quedan definidas las estructuras de nodo funcionales de:

- Un nodo hijo en un Meta4Object multinodo o formulario de información.
- Un nodo raíz en un Meta4Object multinodo o formulario de información.
- El nodo de datos de un Meta4Object de lista y mantenimiento.



Árbol de herencia de Estructuras de nodo de QFB

Todas las estructuras de nodo de filtros QBF funcional deben heredar de la estructura de nodo QBF tecnológico como indica el siguiente diagrama.

Puede comprobar la estructura de nodo padre desde el árbol de herencia, que queda automáticamente habilitada al ejecutar la herencia.



Estructuras de nodo

Los nodos contenidos en un Meta4Object se definen a partir de una estructura de nodo establecida. Se puede decir que el nodo es una instancia de la estructura de nodo o que instancia la estructura de nodo. Una estructura de nodo equivale a una sentencia SQL que se ejecuta en el gestor de base de datos y recupera un conjunto de datos en memoria principal.

A continuación, se describen las estructuras de nodo más importantes de las

plantillas contenidas en el development framework y su funcionalidad:

SRTC_ROOT_TEMPLATE

Ésta es la estructura de nodo raíz para todos los nodos de datos de las plantillas y proporciona la funcionalidad de prevalidación en cliente.

Estructuras de nodo de listas y mantenimientos

SRTC_MT_TEMPLATE

Esta estructura de nodo proporciona la funcionalidad de zoom y prevalidación.

SRTC_TRMT_TEMPLATE

Esta estructura de nodo proporciona la funcionalidad de zoom, validate, list, multiselection y prevalidación.

QBF_TEMPLATE

Esta es la estructura de nodo para todos los nodos QBF y proporciona la funcionalidad de filtrado.

Estructuras de nodo de multinodos y formularios de información

SRTC_MNIN_ROOT_TEMPLATE

Esta estructura de nodo para el nodo raíz de todos los multinodos y formularios de información. Proporciona la funcionalidad de zoom, prevalidación y filtrado de fechas (exclusivamente para los formularios de información).

SRTC_MNIN_CHILD_TEMPLATE

Esta estructura de nodo para todos los nodos hijo raíz de todos los multinodos y formularios de información. Proporciona la funcionalidad de zoom, prevalidación y filtrado de fechas (exclusivamente para los formularios de información).

Metodología de desarrollo avanzada (sin herramientas)

Qué se debe hacer

En el Meta4Object

- Prevalidación. En los Meta4Objects de las plantillas de multinodo y lista y mantenimiento, se propone un mecanismo de validación de datos en cliente antes de grabar. Grabar implica ejecutar el método `persist_tree` e ir al servidor, con lo que si la grabación va a fallar y se puede evitar el viaje al servidor a grabar y la vuelta para indicar que la grabación ha fallado, se optimiza el sistema.

Este mecanismo consiste en crear un método de usuario que haga el control que se desee en la parte cliente, y en caso de no querer lanzar la persistencia, se llama a la función `EndVMTransaction(M4_ROLLBACK_RESUME)` y no `M4_ROLLBACK` por si se necesitara recuperar el control de la ejecución.

Este método de usuario debe estar conectado con tipo de precedencia Antes e In-site, con el método de plantilla `PREVALIDATE_PRG`.

- Intentar hacer las mínimas modificaciones en el objeto padre. Hay algunas obligadas por las limitaciones de herencia, pero sólo hacerlo cuando no queda más remedio.
- Si en un objeto existe un método donde añadir nuestra localización, sobrescribir la regla de ese método.
- Utilizar los mapeos de Meta4Object y la asociación al árbol de ejecución para despreocuparnos de las referencias a ese Meta4Object
- Modificar la parametrización C/S sólo en aquellos métodos que necesitamos forzar que vayan al servidor (básicamente los que van a ejecutar cargas y grabaciones)
- Control de obligatoriedad en cliente. Asignando la propiedad 'No nulo' en el elemento se comprueba automáticamente que ese elemento tiene un valor antes de ir al servidor (con esto se evitan viajes al servidor innecesarios).
- Reducir al mínimo número de líneas posible la sobreescritura de métodos.
- Si necesito sobrescribir una regla LN4, si las líneas que quiero añadir están localizadas en bloques, valorar si no sería mejor sobrescribir ese código añadiendo sólo la llamada a un nuevo método que creamos donde añadimos toda nuestra nueva funcionalidad. Esto facilitará la integración en el usuario de las actualizaciones.
- En el caso en que se quiera modificar la sentencia de filtro de un filtro QBF en un Meta4Object de mantenimiento y no se pueda hacer con la herramienta (una sentencia con un `EXISTS` o una sentencia que queremos para filtrar pero que no queremos traer la información), habría que:

- En la EN del QBF, en los conceptos que sea necesario (PVT_GET_FROM o PVT_GET_WHERE), crear una nueva regla (con ID distinto de PVT_GET_FROM_EX y PVT_GET_WHERE_EX ya que este ID lo utiliza la herramienta) y con el orden que se desee, con la sentencia que se quiere añadir al filtro.

De esta forma se garantiza que el Meta4Object se puede seguir editando con la herramienta añadiendo campos de la forma habitual sin que se pierda esta modificación.

En la presentación

Trabajar con los elementos dentro de paneles y no con los elementos directamente en el Mimic. Se asigna a los paneles la propiedad Align ='top'; son mucho más fáciles de manejar para la posición y para visibilizar/invisibilizar (se ajustan automáticamente)

Heredar los includes que se quieren modificar (no se pueden mapear pero se modifica la llamada en la presentación que lo contiene y ya está)

Utilizar los mapeos de presentaciones para despreocuparse de las referencias a esa presentación.

Modificar comportamientos de los includes genéricos tecnológicos sin heredarlos.

Por ejemplo, invisibilizar botones de los objetos de la toolbar añadiendo en el evento *Evload del Tab correspondiente al nodo el siguiente código.*

```
BEGIN Action_set Action_set2
  Set = "**O*../inclpaNodeToolbar/tlbNode/tlbToolbardatagroup.Buttonnew:=0"
END
```

Qué no se debe hacer

En el Meta4Object

En los Meta4Objects de mantenimiento:

- En el nodo QBF no se debe tocar la regla PVT_GET_FROM_EX del concepto PVT_GET_FROM ni la regla PVT_GET_WHERE_EX del concepto PVT_GET_WHERE ya que estas dos reglas son las que gestiona la herramienta (ni borrarlas ni modificar su código, ese código se edita con la herramienta). Si se desea modificar la sentencia del filtro la forma de hacerlo se indica en el apartado 'Qué se debe hacer'.

En la presentación

Aunque técnicamente se puede hacer, no conviene borrar objetos que vienen del padre, es preferible hacerlos no visibles si es posible.

No se deben modificar las rutas o los alias de los objetos de presentación porque se perdería el mantenimiento con las herramientas del framework. Para localizar el código OBL donde se debe añadir la modificación realizada en el Meta4Object, la herramienta busca los objetos con un identificador determinado.

Transferencia de datos entre Meta4Objects

Visión general

El development framework aporta una nueva forma de comunicación entre Meta4Objects. Se realizará a través de un Meta4Object (objeto de transferencia) que a partir de una parametrización va a ser capaz de relacionar un Meta4Object origen con uno o más Meta4Objects destino de la siguiente manera:

- Se parametriza como origen el Meta4Object que manda realizar las acciones (un ejemplo sería el caso del alta) y como destino el que realmente las ejecuta (en el ejemplo serían los mantenimientos). Esta parametrización debe soportar:
 - Traspaso de datos de origen a destino y viceversa
 - Inserción de datos en destino
 - Actualización de datos en destino
 - Borrado de datos en destino
 - Ejecución de métodos tanto en origen como en destino
 - Interrelación entre las acciones anteriores

Esta transferencia se usa principalmente en la funcionalidad **Alta de la persona**.

Parametrizar la transferencia

Para realizar la parametrización asociada a un Meta4Object origen y uno o varios destinos necesitará crear procesos de transferencia que son acciones (traspaso de datos, ejecuciones...) entre los Meta4Object. Cada proceso de transferencia está formado por relaciones que se ejecutan en un orden y sentido predeterminado.

Una relación está formada por distintos tipos de elementos que pueden ser o elementos de relación (relación entre elementos del Meta4Object origen y del destino o de un valor constante) o ejecución de elementos o un operador (indica la acción a realizar: insertar, actualizar, borrar o cerrar, borrar.)

Llamada al Meta4Object de transferencia

El código de la llamada será el siguiente:

```
SCH_M4O_TRANSFER!SCH_LOAD_T3_CATEGORY.EXECUTE_T3_CATEG
ORY(sIdM4O, nHandle , IdFlow , nClientServer)
```

Indicando:

- sIdM4O: ID Meta4Object origen
- nHandle: ID de la instancia del Meta4Object origen
- IdFlow: ID Proceso de transferencia a ejecutar
- nClientServer: Indica donde se desea realizar la ejecución del proceso completo (2-Server, 3-Both)

Consideraciones del Meta4Object de transferencia

Una vez cargada la parametrización definitiva, se irá ejecutando el traspaso de datos siguiendo el orden calculado previamente.

Todas las acciones que se realicen en un proceso de transferencia estarán contenidas en una única **transacción**. En el caso de fallo en cualquiera de los elementos todo el proceso de transferencia fallará sin perjuicio de la integridad de datos.

En cuanto al **sentido de ejecución**. En la definición de la asociación de la relación al proceso de transferencia se indica, para el caso de traspaso de datos, si el traspaso es de 'Envío de datos' o 'Recuperación de datos' y se hará el traspaso según esto.

Con respecto a la ejecución de métodos, se ejecutarán como se indica (independientemente del sentido), si la relación tiene métodos del Meta4Object origen y métodos del Meta4Object destino, se ejecutarán en destino los del destino y en origen los del origen. Por lo tanto, una relación que no tiene definido ningún 'elemento de relación' y que es todo de 'ejecución de elementos' se comporta igual tanto si está definido como de 'Envío de datos' como de 'Recuperación de datos'.



Tiene disponible un mecanismo de traza el cual el sistema informa de las operaciones que se están realizando con el fin de facilitar las tareas de depuración de errores.

Proceso del Meta4Object de transferencia

Gestión de Meta4Object destino e instancias

Se evalúa con qué Meta4Object destino deseo trabajar y si hay una instancia ya levantada, uso dicha instancia y si no, se levanta una nueva.

Cada vez que cambio de relación en el proceso compruebo si estoy cambiando también de Meta4Object destino o no para ver si tengo que buscar una instancia ya existente o no.

Decisión de ejecución de relaciones.

Para decidir si se ejecuta o no una relación, para cada relación en el proceso, se considera:

- Decisiones por nodo:
 - Nodo sin registros. Si el nodo no tiene registros, se analiza la ejecución de la relación una única vez.
 - Nodo con registros: Si tiene registros, el análisis de la ejecución de la relación se hará para cada uno de ellos.

Por cada registro se decide si se ejecuta la relación o no y la acción a realizar dependiendo de sus datos. Si se quiere realizar la misma acción para todos los registros del bloque, se tendrán que crear de ámbito bloque los elementos que se utilizan para decidir la acción a realizar. Si queremos tratar varios registros a la vez es conveniente crearlos de tipo registro.

Antes de ejecutar una relación en la que se relacionan elementos ya se tiene que saber el número de registros con el que se debe trabajar. Esto significa que no se puede realizar la carga de un nodo y la relación de elementos dentro de la misma relación. **La carga (o la creación de registros por parte del usuario) debe ir en una relación previa para que la relación que relaciona elementos ya sepa con cuantos registros debe interactuar** (tanto para el envío de datos como para la recuperación de datos): generar una relación con carga de datos y otra para relación entre elementos

- Decisiones por registro. Se evalúa si hay o no que ejecutar la relación (para cada registro en particular en caso de que haya registros)
 - Si el sentido es de 'Recuperación de datos', la relación se ejecuta siempre.
 - Si el sentido es de 'Envío de datos':

- Si no tiene ningún elemento de relación asociado (o sea, son todo ejecuciones de elementos) se ejecuta siempre.
- Si tiene elementos de relación asociados, entonces:
 - Se revisa si se indica o no una operación a ejecutar y en este caso qué operación es (0-Insertar, 1-Actualizar, 2-Borrar o cerrar, 3-Borrar).
 - Se transferirán todos los elementos de esa relación (ya sean de un nodo o de n) si hay definido algún campo 'obligatorio' con valor no nulo o si se está mandando una operación de 'borrar' o 'borrar o cerrar'. La propiedad de 'obligatorio' sólo se utiliza para esto.

Así que si en una relación sólo se definen elementos como claves primarias, el desarrollador tendrá que crear un campo (una propiedad) que defina como 'obligatorio' y que le permita gestionar esto. **Si en una relación no hay ningún elemento definido como 'obligatorio' se considerará que la parametrización es errónea.**

Acciones a realizar

Una vez se ha decidido que esa relación se va a ejecutar, para cada relación (y en el caso de multiregistro para cada registro) se decide la acción a realizar. Esto se haría de la siguiente manera:

- Si en la parametrización se indica la ejecución de un elemento en origen o en destino es allí donde se ejecuta.

Si el resultado de esa ejecución coincide con el indicado en la propiedad 'valor del error' de la parametrización, se marca la transacción como errónea para que al finalizar la ejecución del proceso, se aborte la transacción de todo el proceso de transferencia (de esta forma aparecen de una vez todos los errores de validación por ejemplo).

Si en la relación se mezclan ejecuciones de métodos y elementos, se ejecutan los métodos por cada registro que se encuentre en origen (si es dirección 'Envío de datos') o destino (si es dirección 'Recuperación de datos')

- Si en la parametrización se indican los elementos de relación, el proceso genera una agrupación de elementos que tenga sentido para realizar la comunicación.

En principio se va a hacer una transferencia de la relación por nodo en destino, esto es, agrupando todos los elementos (con el mismo sentido) correspondientes al mismo nodo destino, esto permitirá montar una o n búsquedas y una o n inserciones, modificaciones o borrados según el sentido de ejecución y los datos.

Hay dos propiedades a tener en cuenta en la gestión de los datos, que son si el elemento es 'CP' o es 'CP sólo para recuperación de datos'.

En cuanto al tratamiento de fechas:

- Si la tabla destino es temporal y no está parametrizada la fecha inicio en la parametrización, trabajo con la fecha de hoy como fecha de inicio.
- Si la fecha fin no está parametrizada o está parametrizada pero tiene valor nulo, se asigna la fecha de PLUS_INF como fecha fin.

Para cada nodo destino y para cada registro en origen:

En destino se tendrá cuidado para cargar (si se quiere borrar o actualizar) un único registro que cumpla las claves primarias que será con el que se quiere trabajar.

- Si el sentido de la relación es de 'Recuperación de datos':
 - Se toma sólo el valor de las 'CPs' y 'CP sólo para recuperar datos' con valores no nulos para localizar el registro.
 - Si se encuentra un registro en destino que cumple las condiciones anteriores, se van pasando los datos de ese registro del destino al origen y sólo se añadirá registro en origen cuando sea necesario (cuando el elemento de origen esté definido de ámbito registro), en este caso: Si en origen tengo algún elemento de ámbito registro y puede recibir varios registros, se pasarán todos los del destino que cumplan la condición de búsqueda (campos de la CP y CP sólo para recuperar datos) al origen añadiendo en origen tantos registros como registros en destino cumplan la condición de búsqueda. Si en origen no hay ningún elemento de ámbito registro, sólo se trae del destino el primer registro que cumpla las condiciones de búsqueda.
- Si el sentido de la relación es de 'Envío de datos':
 - Se busca (a través de la parametrización) el tipo de operación que se desea aplicar a los datos que se están tratando. Si no se manda operador, se aplicará el de por defecto. Si el nodo origen es multiregistro, el operador se evalúa para cada uno de estos registros origen.
 - Se toma sólo el valor de las CPs con valores no nulos para localizar el registro. Si alguna de las claves primarias tuviera valor nulo, no se puede localizar un registro, por lo que sólo tendría sentido la operación de inserción.
 - Si se parametriza el operador. El operador indicará la acción a realizar para todos los nodos parametrizados en la relación. Si no se parametriza operador o éste va vacío, la acción se deducirá según el nodo destino.
 - 0-Insertar: Se inserta un nuevo registro con lo que se manda. Sólo se añade un nuevo registro en destino cuando es necesario (o sea, cuando hay algún elemento de ámbito registro)

- 1-Actualizar: La actualización se comporta igual que la inserción si todos los elementos son de ámbito bloque o nodo. Si manda toda la clave primaria y encuentra registro -> se actualiza. Si no manda toda la clave primaria o no encuentra registro en destino, no hace nada.
- 2-Borrar o cerrar: La fecha inicio que se pasa a la parametrización se considerará como fecha de trabajo, esto es, una operación de borrado se transformará en un cierre de registro o no dependiendo de esta fecha y la del registro actual y de la temporalidad de la tabla. Además, en caso de que la acción final sea un cierre de registro, esta fecha de cierre sería esa (fecha de trabajo-1).
- Si manda toda la clave primaria y encuentra registro -> o se borra o se cierra de registro dependiendo de la temporalidad de la tabla y de la fecha de inicio que se manda (no se tiene en cuenta la fecha de fin). Si la tabla es eterna, borro el registro. Si la tabla tiene fecha (1P o NP):
 - Si la fecha inicio que estoy mandando permite cerrar el registro (fecha inicio que mando -1 >= fecha inicio del registro en destino), se cierra.
 - Si la fecha inicio que mando no permite cerrar el registro (fecha inicio que mando -1 < fecha inicio del registro en destino), se borra.
- Si no manda toda la clave primaria o no encuentra registro en destino, no hace nada.
- 3-Borrar: Si manda toda la clave primaria y encuentra registro se borra. Si no manda toda la clave primaria o no encuentra registro, no hace nada.
- Si no se parametriza operador. Si no se indica ningún campo en la parametrización de tipo operador o si el operador se envía a nulo, dependiendo de los datos, se hará una inserción o una actualización pero nunca un borrado.
- Si viene toda la clave primaria. Creo la sentencia de búsqueda con las claves primarias
 - Si no encuentro registro: insertar (igual que con operador)
 - Si encuentro registro
 - Si los datos son los mismo que los que se mandan, no se hace nada (así se evitan actualizaciones inútiles de datos)
 - Si los datos sólo cambian en las fechas de la temporalidad ' no se hace nada.
 - Si cambia algún otro dato (excepto fechas):
 - Si es eterna o es 1P ' se actualiza pero no se actualiza la información de las fechas.
 - Si es NP: si la fecha inicio es igual ' actualizo lo que me llegue en la parametrización o si la fecha inicio no coincide' inserto

-
- Si no viene toda la clave primaria. Creo un nuevo registro con lo que tengo y asigno los datos que llegan.

Cuando acaba la ejecución del proceso de transferencia ya sea correctamente o con errores, se borran las instancias de los Meta4Objects de destino, a no ser que esté parametrizado en la llamada al método EXECUTE_T3_CATEGORY que la instancias se dejen abiertas.

Caso práctico avanzado

4

Acerca de este capítulo

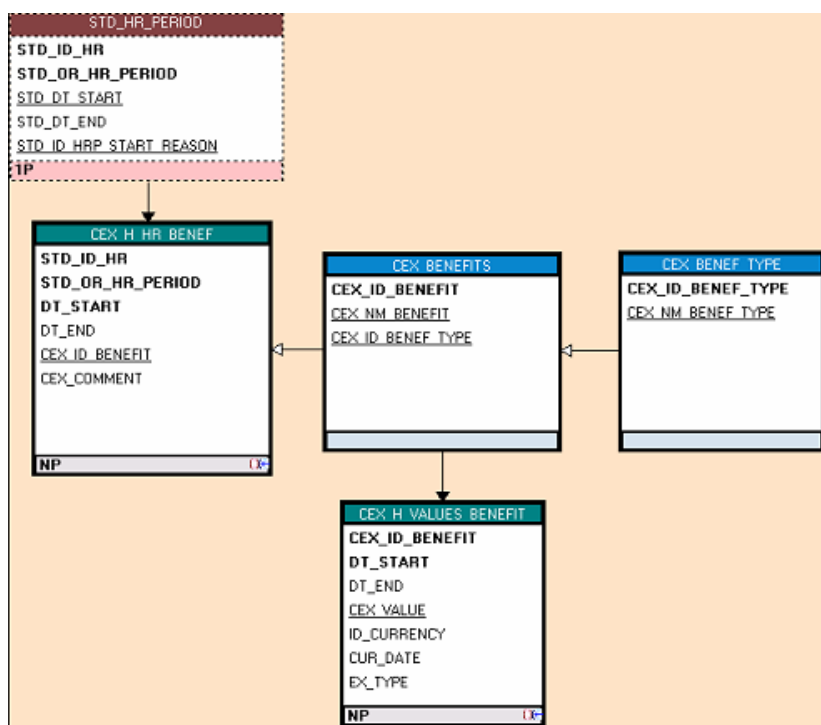
El objetivo es presentar una serie de casos prácticos funcionales avanzados donde se utilizarán todos los conceptos explicados previamente.

Añadir tabla histórica al producto

Se añade la gestión de un histórico que depende del Periodo del RH en:

- El mantenimiento del periodo
- El alta
- La baja
- La cancelación de la baja
- Borrado

Para hacer esto, se utiliza el ejemplo con el que se ha trabajado previamente suponiendo que en el alta sólo se puede asignar un beneficio al empleado.



En este ejemplo se da por hecho que se ha pasado por el ejemplo anterior y ya están creados los multinodos y los mantenimientos del módulo de beneficios para que las presentaciones generen bien los remotes. También se presupone que las modificaciones son las primeras modificaciones que se realizan sobre el objeto estándar. Por esta razón es necesario crear un nuevo objeto que herede del estándar, para garantizar de esta manera el mantenimiento automático del producto estándar en nuestro entorno.

En caso de que esta operación de herencia ya se hubiera realizado, se accederá a la herramienta seleccionando directamente el objeto que hemos creado.

Mantenimiento del periodo

Habría que modificar el Meta4Object y la presentación de Mantenimiento del periodo.

El Meta4Object que se quiere modificar es el Meta4Object estándar SRCO_PA_MN_HR_PERIOD, así que habrá que heredar.

Asistente

Se accede al asistente del development framework.

Se selecciona 'Nuevo multinodo' (porque el padre es un multinodo)

Una vez en el asistente:

- Fase 1 (1 de 4)

Indicamos la acción 'Nuevo heredando de uno existente'

Meta4Object: SRCO_PA_MN_HR_PERIOD

- Fase 1 (2 de 4)

Indicar el identificador del nuevo Meta4Object. En el ejemplo CREX_PA_MN_HR_PERIOD donde:

CREX: Cliente Rich

Indicar el nuevo ID de la EN raíz por si se quiere sobrescribir (aunque no se necesita) que será CREX_PA_MN_HR_PERIOD.

- Fase 1 (3 de 4)

Posicionarse en el nodo raíz (SRCO_PA_WZ_HR_PERIOD)

Definir el nodo hijo del multinodo haciendo clic en el botón 'nuevo'.

- Fase 2 (1 de 1)

Llamar al nuevo nodo CREX_PA_MN_H_HR_BENEF

Del listado de tablas seleccionamos la tabla CREX_PA_MN_H_HR_BENEF

Seleccionar todos los campos y por ser una tabla temporal, ordenar por fecha de inicio descendente (para tener acceso al registro más reciente)

Se crean automáticamente los conectores entre los campos relacionados de las dos tabla (la del nodo raíz y la del nodo hijo) si están relacionadas por modelo de datos. En el ejemplo no es necesario entrar en la definición de conectores porque están definidos en el modelo de datos.

- Fase 1 (4 de 4)

Como se quiere crear la presentación asociada, seleccionamos 'grabar y editar presentación'.

- Editar presentación

- Fase 1 (1 de 2)

Como el Meta4Object original ya estaba utilizando una presentación, el editor de presentaciones ya interpreta que debe heredar la presentación.

El Id es el mismo que el del Meta4Object CREX_PA_MN_HR_PERIOD.

Fase 1 (2 de 2)

Se chequean todos los nodos porque se quiere visualizar todos en la presentación.

Posicionarse en el nodo hijo que acabamos de crear, pulsar el botón 'Editar'.

- Fase 2 (1 de 2)

No se quiere ningún enlace asociado a este nodo.

- Fase 2 (2 de 2)

En el formato tabla queremos el ID del RH, el Num. Periodo, la fecha de inicio, la de fin y el ID del beneficio.

Si se quisiera que el nodo sólo se viera para un tipo de contratación, habría que indicar esto en la presentación. En el ejemplo no se gestiona.

Revisar que en la presentación queda el nodo como lo queremos y grabar.

Editor de Meta4Object avanzado.

Editar el Meta4Object CREX_PA_MN_HR_PERIOD en modo avanzado.

Para que el alta recupere los datos a la fecha correcta, si el nodo es un histórico hay que filtrar por fechas, para conseguir esto habría que:

- Conectar con el nodo padre las propiedades CUT_OFF_DATE
- Modificar la propiedad avanzada del nodo:
 - Filtro automático: Histórico.

Mapeos

Se acaba de crear un nuevo Meta4Object y una nueva presentación y se quiere indicar que se ejecute siempre este nuevo Meta4Object y esta nueva presentación en vez de los objetos estándares.

Esto se indica a través de los mapeos.

Se crea:

- Un mapeo de presentación llamado CREX_PA_MN_HR_PERIOD de la presentación origen SRCO_PA_MN_HR_PERIOD a la destino CREX_PA_MN_HR_PERIOD
- Un mapeo de Meta4Object igual que el de presentación

Entorno de ejecución

Se deberán asignar los mapeos que se acaban de crear al entorno de ejecución que corresponda en el árbol de proyectos.

El alta

Habría que modificar el Meta4Object, la presentación y la parametrización del Alta del empleado.

El Meta4Object que se quiere modificar es el Meta4Object estándar SRCO_PA_MN_HIRE, así que habrá que heredar.

Se considera que el módulo de Benefits es de nómina, por lo tanto en el alta habría que añadirlo en el paso correspondiente a nómina.

Meta4Object

Crear un nuevo Meta4Object CREX_PA_MN_HIRE que hereda del Meta4Object SRCO_PA_MN_HIRE.

El nodo que corresponde a la información de nómina es el SRCO_PA_HIRE_WIZ_PAY. Sobrecribir el nodo porque hay que añadir el elemento para indicar el ID Beneficio. Llamar a la nueva EN: CREX_PA_HIRE_WIZ_PAY.

Creamos un nuevo elemento:

- Tipo de elemento: Propiedad.
- ID: CEX_ID_BENEFIT
- Ámbito: bloque
- Tipo básico: cadena variable (lo mismo que el campo de la tabla)
- Precisión: 10 (lo mismo que el campo de la tabla)
- Visible: Si
- Objeto de lectura: CEX_H_HR_BENEF (esto es para facilitar la creación en la presentación)
- Campo de lectura: CEX_ID_BENEFIT (esto es para facilitar la creación en la presentación)

Presentación

Crear una nueva presentación que hereda de la presentación SRCO_PA_MN_HIRE. La llamaremos CREX_PA_MN_HIRE

Buscar en la presentación el tab que se corresponde con el nodo que se quiere modificar, en el ejemplo el objeto es *O*/Pres2/FormMain/splForm/splForm_Right/splNode/splNode_Down/splNodeFields/splNodeFields_Up/chaChangerNodes/TABSTRIP_ROOTS/TAB_SRCO_PA_HIRE_WIZ_PAY/MimicSRCO_PA_HIRE_WIZ_PAY

En este objeto, crear un nuevo panel donde añadir el nuevo elemento con las propiedades:

- Height = 28
- Align = "Top"
- Borderstyle = Ninguno

En el panel, seleccionar la acción 'asistente de inserción de registro' e indicar el elemento CEX_ID_BENEFIT, con esto se crea la línea completa en la presentación

Revisar la propiedad Top y left de los objetos creados para ajustarlos con el resto de la presentación

Para colocar el panel en el sitio deseado, ir a la pestaña 'Componentes' en las propiedades del Mimic.

Parametrización de visualización de pasos para nuestro nuevo Meta4Object del alta

Como acabamos de crear el Meta4Object del Alta, debemos definir qué información del empleado queremos ver o no según el tipo de empleado.

En la opción de menú de configuración del alta, creo una configuración para el nuevo Meta4Object CREX_PA_MN_HIRE indicando para cada tipo de alta qué pasos queremos ver o no.

Visualización del elemento en la definición de tipos de alta

Si se quisiera que la información de este elemento no se pudiera guardar en los tipos de alta, tendría que indicarlo. En nuestro ejemplo no lo vamos a gestionar.

Mapeos

Tras crear un nuevo Meta4Object y una nueva presentación y se quiere indicar

que en se ejecute siempre este nuevo Meta4Object y esta nueva presentación en vez de los objetos estándares.

Esto lo indicamos a través de los mapeos:

- Un mapeo de presentación llamado CREX_PA_MN_HIRE de la presentación origen SRCO_PA_MN_HIRE a la destino CREX_PA_MN_HIRE
- Un mapeo de Meta4Object igual que el de presentación

Entorno de ejecución

Se deberán asignar los mapeos que acabamos de crear al entorno de ejecución que corresponda en el árbol de proyectos.

Parametrización

En este paso se indica la relación entre los elementos del alta y los elementos del mantenimiento.

- Fase 1 (1 de 1)
 - Indicar la acción *Parametrizar nuevo nodo*
 - ID Meta4Object origen: CREX_PA_MN_HIRE
 - ID Meta4Object destino: CREX_PA_MN_HR_PERIOD
 - ID nodo destino: CREX_PA_H_HR_BENEF
 - ID proceso de transferencia: LOAD_HIRE
- Fase 2 (1 de 2)

Elementos de relación:

- SRCO_PA_HIRE.SRCO_ARG_ID_PERSON - STD_ID_HR
- SRCO_PA_HIRE.SRCO_DT_HIRE_PARAM - DT_START
- SRCO_PA_HIRE_WIZ_PAY.CEX_ID_BENEFIT - CEX_ID_BENEFIT
- Fase 2 (2 de 2)
 - Chequear LOAD_HIRE (recuperación de datos)
 - Chequear SAVE_HIRE (envío de datos)

La baja (parametrización)

Como el histórico no se ha definido como 'cierre en cascade', hay que definir el comportamiento con los registros existentes en caso de dar de baja al

empleado.

Hay que controlar los registros que se ven afectados por la fecha de baja y los registros a future.

- Crear un nuevo registro para la tabla CEX_H_HR_BENEF
 - Resolution automática: Si
 - Tipo: Incited antes de la baja y Finalizada después
 - Dependency: Periodo
 - Tipo de resolution: Cerrado
- Creo un nuevo registro para la tabla CEX_H_HR_BENEF
 - Resolution automática: Si
 - Tipo: Incited después de la baja
 - Dependency: Periodo
 - Tipo de resolution: Borrado

La cancelación de la baja (parametrización)

Crear un nuevo registro para la tabla CEX_H_HR_BENEF

- Tipo de conflict: 1 (depende del periodo)
- Tipo de acción en conflict: 0 (repairer)

Borrado (parametrización)

Como el histórico no se ha definido como 'Borrado en cascade', hay que definir el comportamiento con los registros existentes en caso de borrar al empleado.

Se crea un nuevo registro para la tabla CEX_H_HR_BENEF.

- Nivel de persona: 'Periodo del RH' (no se borrará cuando se borre la persona o el candidato). No es necesario indicar un sustituto para los datos existentes.